

NBA Intelligent Scheduling & Prediction System

Sistema intelligente per predizione, scheduling e ottimizzazione logistica nel dominio NBA

Gruppo di lavoro:

- Francesco_Sparascio, 796395, f.sparascio3@studenti.uniba.it
- Giuseppe_Guanti, 803088, g.guanti1@studenti.uniba.it

Repository GitHub: [giuseppiguanti-wrld/NBA-Intelligent-Scheduling-Prediction-System](https://github.com/giuseppiguanti-wrld/NBA-Intelligent-Scheduling-Prediction-System)

A.A. 2025/2026

Indice

1	Introduzione	3
1.1	Dominio di interesse	3
1.2	Knowledge Based System	3
1.3	Origine dei dati	3
1.4	Costruzione della conoscenza	5
2	Analisi Esplorativa dei Dati	7
2.1	Sommario	7
2.2	Fondamenti teorici dell'EDA	7
2.3	Modellazione del problema nel dominio NBA	7
2.4	Implementazione dell'analisi	8
2.5	Strumenti utilizzati	8
2.6	Risultati e interpretazione	8
2.6.1	EDA univariata	8
2.6.2	EDA bivariata	12
2.7	Discussione critica	17
3	Feature Engineering e Modellazione	19
3.1	Sommario	19
3.2	Fondamenti teorici del Feature Engineering	19
3.3	Modellazione del problema	19
3.4	Implementazione	20
3.5	Strumenti utilizzati	20
3.6	Decisioni di progetto	20
3.7	Risultati	21
3.8	Valutazione e discussione critica	21
4	Addestramento e Validazione del Modello	22
4.1	Sommario	22
4.2	Fondamenti teorici del Machine Learning supervisionato	22
4.3	Modellazione del problema NBA	23
4.4	Ensemble Learning	23
4.5	Strumenti utilizzati	25
4.6	Pipeline e decisioni di progetto	25
4.6.1	Baseline	26
4.6.2	Walk-Forward Cross Validation	26
4.7	Risultati di validazione walk-forward	27
4.8	Risultati di classificazione sul test set	27
4.9	Risultati di regressione	29

4.10	Feature Importance e interpretabilità	30
4.11	Scelta finale del modello	32
5	Modellazione CSP	33
5.1	Sommario	33
5.2	Knowledge Base	33
5.2.1	Rappresentazione della conoscenza	34
5.3	Modellazione CSP	34
5.3.1	Vincoli hard	34
5.3.2	Vincoli business	35
5.3.3	Formalizzazione matematica	35
5.3.4	Analisi dello schedule	35
5.4	CP-SAT e ragionamento	36
5.4.1	Complessità del problema	36
5.4.2	Estensione a vincoli soft	37
5.5	Valutazione e discussione critica	37
5.5.1	Limiti della modellazione CSP	37
6	Ricerca Informata e ottimizzazione logistica	38
6.1	Sommario	38
6.2	Fondamenti teorici della ricerca informata	38
6.3	Modellazione dello spazio degli stati	39
6.4	Algoritmo A*	39
6.5	Euristica MST	39
6.5.1	Analisi dell'euristica	40
6.6	Ottimizzazioni	40
6.6.1	Confronto con strategie non informate	41
6.6.2	Analisi del pruning	42
6.6.3	Interpretazione logistica	42
6.7	Valutazione e discussione critica	42
6.7.1	Limiti metodologici e sviluppi	42
7	Conclusioni	44
7.1	Sviluppi Futuri	44

Capitolo 1

Introduzione

1.1 Dominio di interesse

Il progetto *NBA Intelligent Scheduling & Prediction System* studia il dominio NBA come problema integrato di conoscenza, previsione e decisione. Una stagione NBA contiene migliaia di eventi sportivi distribuiti nel tempo, vincolati da disponibilità delle arene, trasferte, recupero fisico, diritti televisivi, rivalità sportive e priorità commerciali. Il calendario non è quindi un semplice elenco di partite: è una struttura decisionale in cui ogni assegnazione temporale può influenzare audience, fatica delle squadre e qualità competitiva.

Il sistema affronta tre aspetti complementari. Il modulo di apprendimento supervisionato prevede l'esito della squadra di casa e il differenziale punti usando informazioni storiche e rolling statistics. Il modulo CSP/COP formalizza lo scheduling televisivo come problema di soddisfacimento e ottimizzazione di vincoli. Il modulo di ricerca informata modella i road trip come problema TSP su arene NBA, risolto con A* e un'euristica basata su MST.

1.2 Knowledge Based System

La base di conoscenza del progetto non è un archivio passivo, ma un insieme di rappresentazioni operative. I dati storici NBA generano feature di forma, ritmo, efficienza e riposo; queste feature alimentano i modelli predittivi e diventano attributi simbolici per il CSP, ad esempio nell'identificazione di *big match*. Il modulo A* produce conoscenza logistica sui costi di trasferimento. In prospettiva, tale costo può diventare un vincolo o una penalità nello scheduling.

1.3 Origine dei dati

La sorgente utilizzata per il progetto è `nba_api`, in particolare gli endpoint `LeagueGameFinder`, `BoxScoreTraditionalV3` e `BoxScoreAdvancedV3`. La copertura temporale va dalla stagione 2000–01 alla stagione 2025–26.

Data Assessment. Il notebook `02_data_assessment.ipynb` analizza il file grezzo `data/1_raw/nba_stats_2000-01_2025-26.csv`, composto da 125,012 righe e 81 colonne. La fase è diagnostica e non modifica i dati: verifica schema, completezza, unicità, validità e consistenza. L'assessment rileva 3 mismatch di tipo rispetto allo schema atteso, 0

colonne critiche per valori mancanti, 0 righe duplicate esatte, 0 duplicati sulla business key, 0 problemi di validità e 0 problemi di consistenza. I risultati definiscono il piano di remediation del notebook successivo.

Data Cleaning. Il notebook `03_data_cleaning.ipynb` produce il file `cleaned_nba_data_2000-01_2025-26.csv` nella directory `data/2_processed/`. Le operazioni effettuate sono: verifica dell'hash del raw CSV, derivazione della variabile `home_away` da `MATCHUP`, rimozione di 62 colonne strutturali, ridondanti o non rilevanti per il modello, deduplicazione delle righe generate dallo split Starters/Bench dell'API, conversione dei tipi, rinomina in `snake_case`, riordino dello schema, controllo di record fisicamente impossibili, normalizzazione difensiva delle stringhe e gestione dei valori mancanti obbligatori. Il cleaning rimuove 62,506 righe duplicate, 0 record invalidi e 0 record con campi obbligatori mancanti; il dataset pulito finale contiene 62,506 righe e 20 colonne. Il dataset finale con feature contiene 31,160 partite e 58 colonne.

Tabella 1.1: Sintesi quantitativa dei dataset del progetto.

Dataset	Righe	Colonne	Stagioni	Ruolo
Raw nba_stats	125 012	81	26	Box score tradizionali e avanzati
Cleaned	62 506	20	26	Una riga per partita validata
Feature engineered	31 160	58	26	Feature rolling e target ML

Tabella 1.2: Numero di partite presenti nel dataset feature engineered per stagione.

Stagione	Partite
2000-01	1 111
2001-02	1 189
2002-03	1 184
2003-04	1 188
2004-05	1 230
2005-06	1 230
2006-07	1 230
2007-08	1 230
2008-09	1 230
2009-10	1 230
2010-11	1 230
2011-12	990
2012-13	1 229
2013-14	1 230
2014-15	1 230
2015-16	1 230
2016-17	1 230
2017-18	1 230
2018-19	1 230
2019-20	1 059
2020-21	1 080
2021-22	1 230
2022-23	1 230
2023-24	1 230
2024-25	1 225
2025-26	1 225

Tabella 1.3: Valori mancanti prima e dopo le fasi di pulizia e feature engineering.

Dataset	Valori mancanti	Percentuale
Raw	2	0.000
Cleaned	0	0.000
Feature engineered	3 653	0.202

1.4 Costruzione della conoscenza

La pipeline segue quattro fasi: ingestion, assessment, cleaning e feature engineering. L'ingestion scarica e consolida le statistiche per stagione; l'assessment verifica dimensioni, duplicati, colonne mancanti e copertura temporale; il cleaning costruisce una rappresentazione a livello partita; il feature engineering aggiunge rolling window e target.

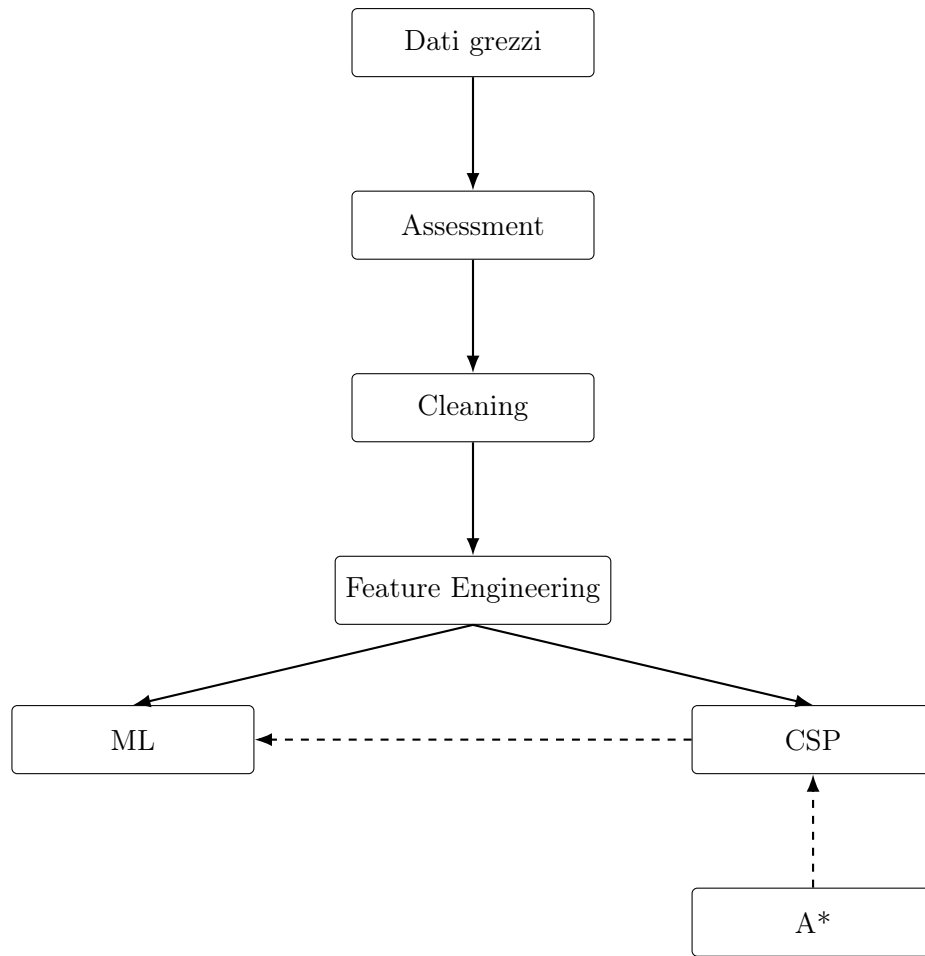


Figura 1.1: Pipeline integrata del KBS: dati, feature, apprendimento, scheduling e ottimizzazione logistica. Le frecce tratteggiate rappresentano eventuali sviluppi futuri, spiegati alla fine.

Capitolo 2

Analisi Esplorativa dei Dati

2.1 Sommario

L'EDA verifica se la conoscenza costruita dai dati sia coerente con il dominio NBA. Le analisi univariate studiano distribuzioni di punteggi, vittorie, differenziali e statistiche avanzate; le analisi bivariate valutano correlazioni, separabilità del target e possibili relazioni non lineari tra forma recente e risultato.

2.2 Fondamenti teorici dell'EDA

L'Analisi Esplorativa dei Dati, o Exploratory Data Analysis, è la fase in cui il dataset viene sintetizzato prima della modellazione [1]. L'obiettivo è individuare distribuzioni, anomalie, relazioni e pattern utili per capire se i dati siano coerenti con il dominio e adatti a sostenere analisi successive.

L'EDA resta descrittiva: non dimostra causalità, ma evidenzia associazioni e criticità. Nel progetto NBA serve soprattutto a controllare range e outlier, osservare il fattore campo, distinguere analisi univariata e bivariata e motivare l'uso di feature temporali e split cronologici.

2.3 Modellazione del problema nel dominio NBA

Nel progetto, l'EDA modella il dataset NBA `cleaned_nba_data_2000-01_2025-26.csv` come una base di conoscenza empirica composta da partite, squadre, stagioni, statistiche tecniche e variabili di calendario. Ogni riga del dataset cleaned rappresenta una partita con attributi relativi alla squadra di casa, alla squadra ospite e al contesto temporale. L'obiettivo dell'EDA è verificare se queste variabili siano coerenti con la conoscenza cestistica: le distribuzioni devono avere range plausibili, i target devono essere interpretabili e le relazioni tra feature devono rispettare dinamiche note del gioco.

Il dominio NBA presenta alcune caratteristiche specifiche. Innanzitutto, la partita è un evento competitivo bilaterale: ogni prestazione ha senso in relazione all'avversario. In secondo luogo, le squadre evolvono nel tempo: forma recente, streak di vittorie, riposo e back-to-back possono modificare la probabilità di vittoria. Infine, esistono cambiamenti storici di lungo periodo: il ritmo e l'efficienza offensiva delle stagioni recenti non coincidono necessariamente con quelli dei primi anni 2000. L'EDA deve quindi studiare sia distribuzioni globali sia differenze temporali.

Un elemento specifico è il fattore campo. La variabile `home_win` non rappresenta soltanto un’etichetta binaria, ma incorpora un fenomeno noto nello sport professionistico: la squadra di casa beneficia di familiarità con l’arena, minore fatica di viaggio e supporto del pubblico.

2.4 Implementazione dell’analisi

L’implementazione dell’EDA segue una logica progressiva. La prima fase calcola statistiche descrittive sulle principali variabili numeriche e verifica la presenza di valori mancanti. La seconda fase produce distribuzioni univariate e boxplot per identificare code, asimmetrie e outlier. La terza fase analizza relazioni bivariate tramite correlazioni, scatter plot e confronti condizionati dal target. La quarta fase osserva pattern stagionali e variabili di calendario, così da collegare l’analisi dati ai successivi moduli di feature engineering e scheduling.

2.5 Strumenti utilizzati

L’analisi è stata condotta con Pandas, NumPy, Matplotlib e Seaborn. Le visualizzazioni sono selezionate per documentare distribuzioni, outlier, andamento temporale e relazioni tra variabili, con attenzione al loro impatto sulla modellazione predittiva.

2.6 Risultati e interpretazione

Tabella 2.1: Statistiche descrittive delle principali feature numeriche.

Feature	Media	Dev.Std	Min	Mediana	Max
<code>point_differential</code>	2.716	13.713	-57.000	4.000	73.000
<code>home_win</code>	0.585	0.493	0.000	1.000	1.000
<code>home_winrate_last_10</code>	0.498	0.209	0.000	0.500	1.000
<code>away_winrate_last_10</code>	0.502	0.209	0.000	0.500	1.000
<code>home_rest_days</code>	4.709	22.404	1.000	2.000	932.000
<code>away_rest_days</code>	4.199	20.715	1.000	2.000	287.000

La Tabella 2.1 riporta le statistiche descrittive prodotte sui principali campi numerici. Il `point_differential` ha media pari a 2.716, mediana pari a 4.000 e deviazione standard pari a 13.713, mostrando una variabilità ampia del margine finale. La variabile `home_win` ha media 0.585, mentre le win rate sulle ultime dieci partite sono bilanciate attorno a 0.5 per squadra di casa e squadra ospite. I giorni di riposo hanno mediana pari a 2, ma presentano valori massimi elevati, evidenziando la presenza di osservazioni estreme da considerare nelle analisi successive.

2.6.1 EDA univariata

L’analisi univariata valuta separatamente le distribuzioni delle variabili principali. In questa fase l’obiettivo non è ancora spiegare il risultato della partita, ma verificare range, forma delle distribuzioni, outlier e coerenza statistica delle variabili.

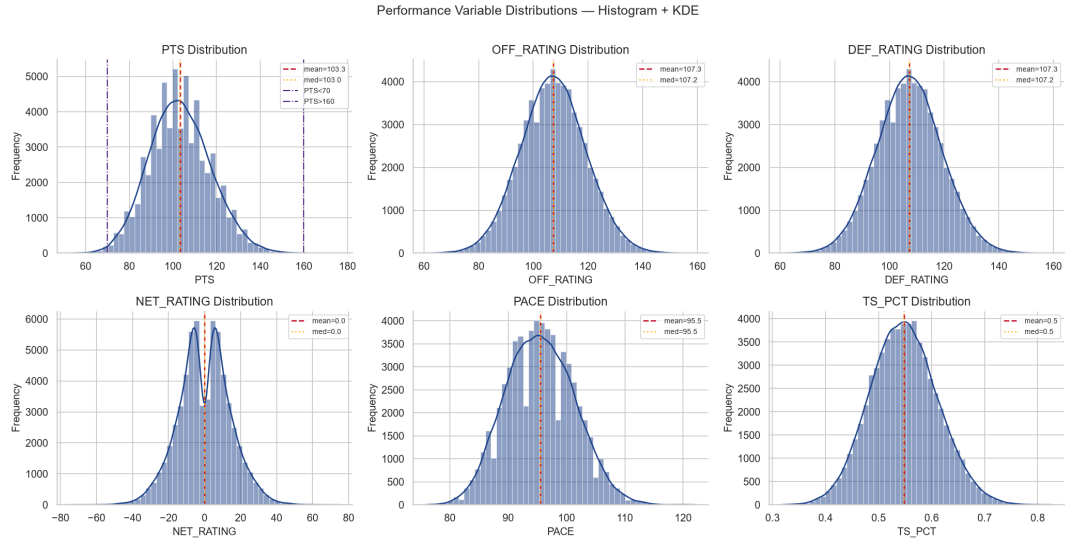


Figura 2.1: Distribuzioni univariate di punti, rating offensivo/difensivo, net rating, pace e true shooting percentage.

La Figura 2.1 mostra distribuzioni concentrate attorno a valori medi realistici per il basket NBA. La simmetria di molte variabili suggerisce che la standardizzazione e le differenze home-away siano scelte adatte; le code, invece, segnalano partite estreme che i modelli devono gestire senza trattarle automaticamente come errori.

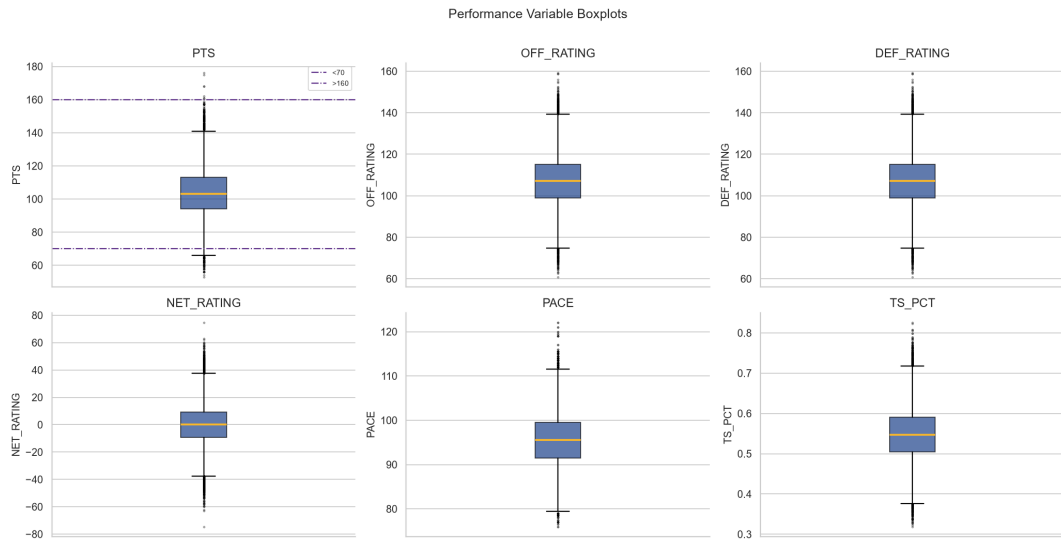


Figura 2.2: Boxplot delle principali variabili prestazionali e identificazione degli outlier.

La Figura 2.2 confronta tramite boxplot la dispersione delle principali variabili prestazionali. I punti segnano una distribuzione concentrata nella fascia centrale, con osservazioni estreme oltre le soglie evidenziate nel grafico; **OFF_RATING** e **DEF_RATING** presentano intervalli simili, mentre **NET_RATING** è distribuito attorno allo zero con code positive e negative. Anche **PACE** e **TS_PCT** mostrano valori fuori dai baffi, utili per individuare partite statisticamente atipiche.

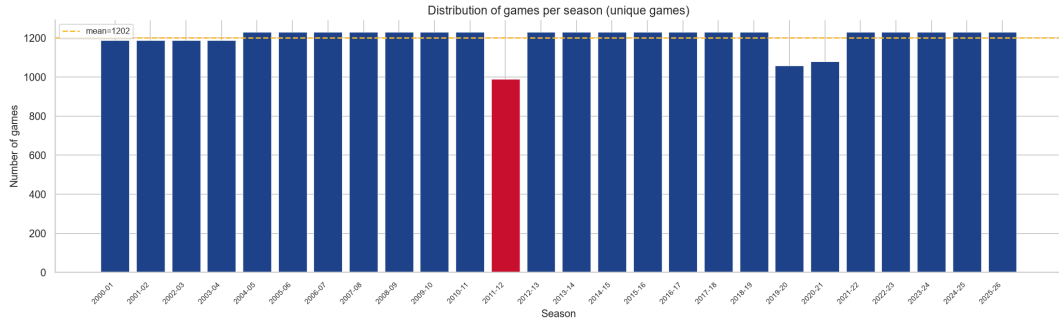


Figura 2.3: Distribuzione del target di classificazione `home_win`.

La Figura 2.3 documenta lo sbilanciamento moderato a favore della squadra di casa. Questo giustifica l'uso di baseline maggioritaria e metriche oltre l'accuracy, come F1, precision, recall e ROC-AUC.

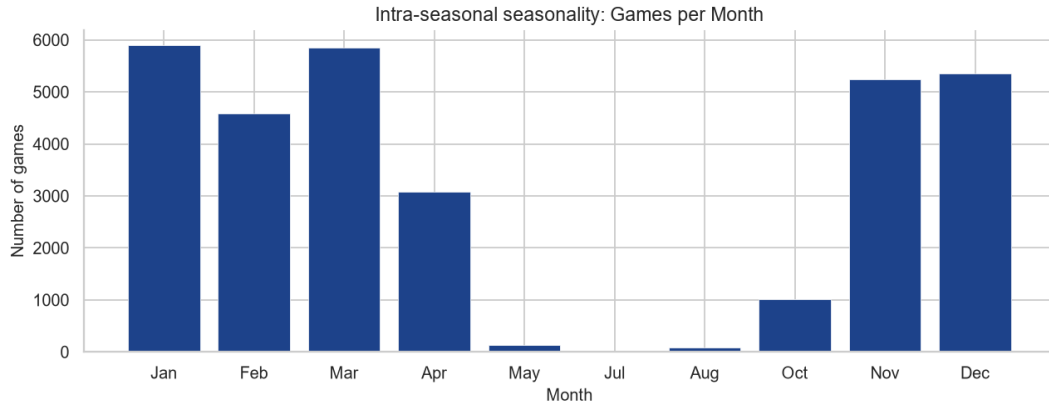


Figura 2.4: Distribuzione del differenziale punti tra squadra di casa e squadra ospite.

La Figura 2.4 mostra un target regressivo centrato vicino allo zero ma con code ampie. La previsione del margine è quindi più informativa della sola classificazione, ma anche più rumorosa per la presenza di scarti estremi.

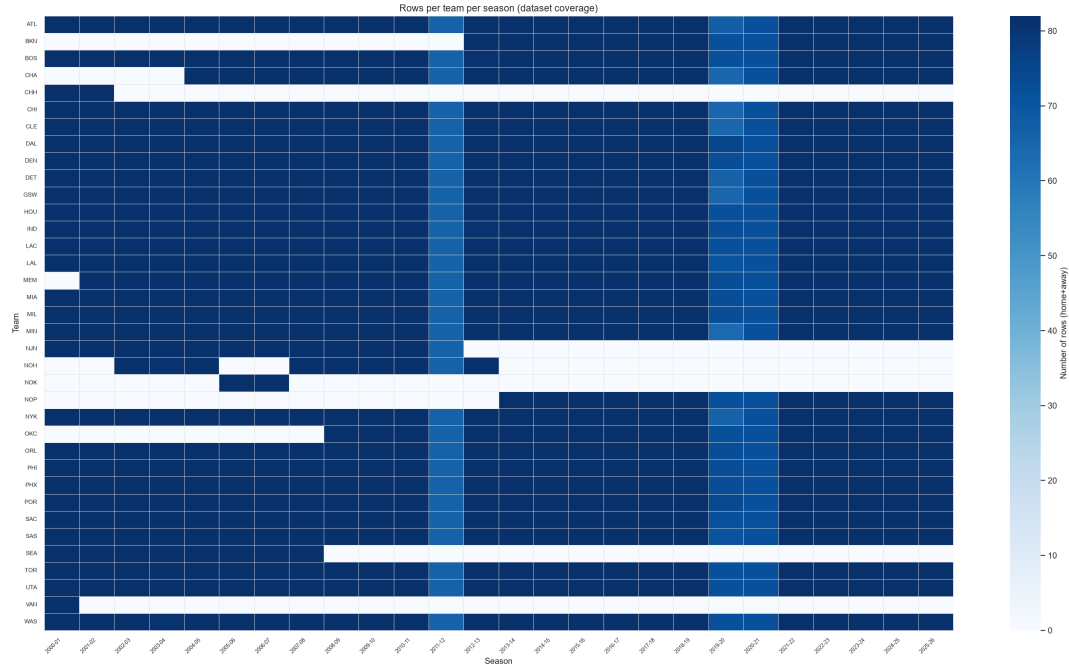


Figura 2.5: Andamento temporale delle variabili NBA lungo le stagioni considerate.

La Figura 2.5 consente di osservare cambiamenti strutturali nel tempo, come evoluzione del pace e dell'efficienza. Questo è il motivo per cui la validazione temporale è preferibile a una validazione casuale.

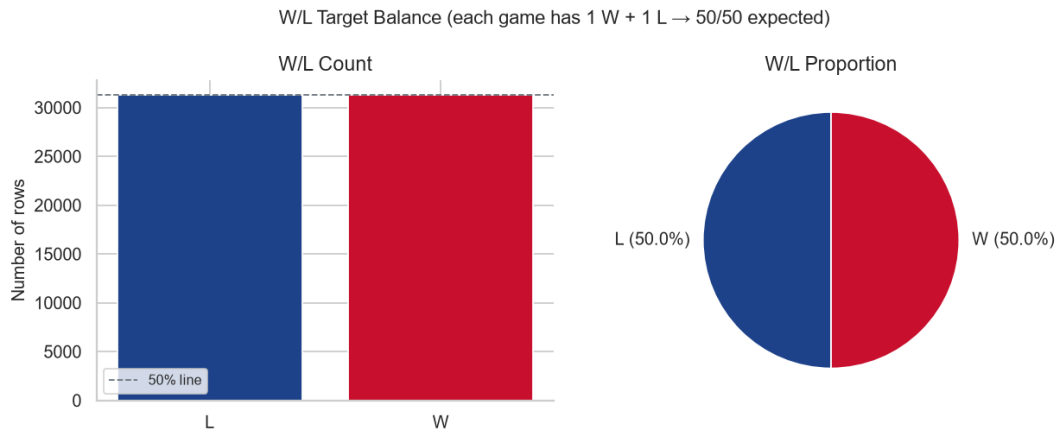


Figura 2.6: Verifica del bilanciamento tra vittorie e sconfitte nel dataset.

La Figura 2.6 verifica che il numero di osservazioni con esito W sia uguale al numero di osservazioni con esito L. Poiché ogni partita deve produrre una vittoria e una sconfitta, uno sbilanciamento tra le due classi indicherebbe perdita o duplicazione di record per le squadre che hanno vinto o perso determinate partite.

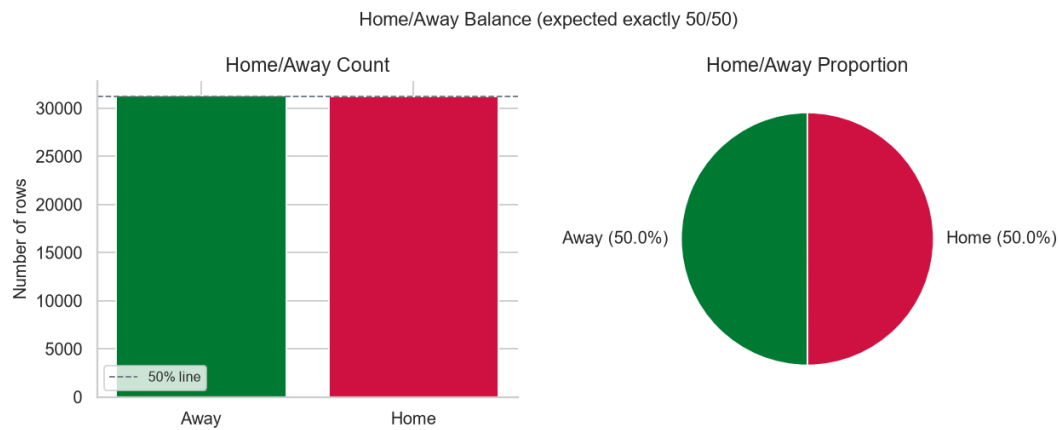


Figura 2.7: Verifica del bilanciamento tra osservazioni home e away.

La Figura 2.7 verifica che le osservazioni **Home** e **Away** siano perfettamente bilanciate. Poiché ogni partita deve comparire una volta per la squadra di casa e una volta per la squadra ospite, la proporzione attesa è 50% e 50%; uno scostamento indicherebbe una perdita o duplicazione di record nella costruzione del dataset.

2.6.2 EDA bivariata

L'analisi bivariata inizia dalla matrice di correlazione e studia relazioni tra feature e target. Qui l'attenzione si sposta dalla qualità della singola variabile alla presenza di segnale predittivo, interazioni e possibili ridondanze.

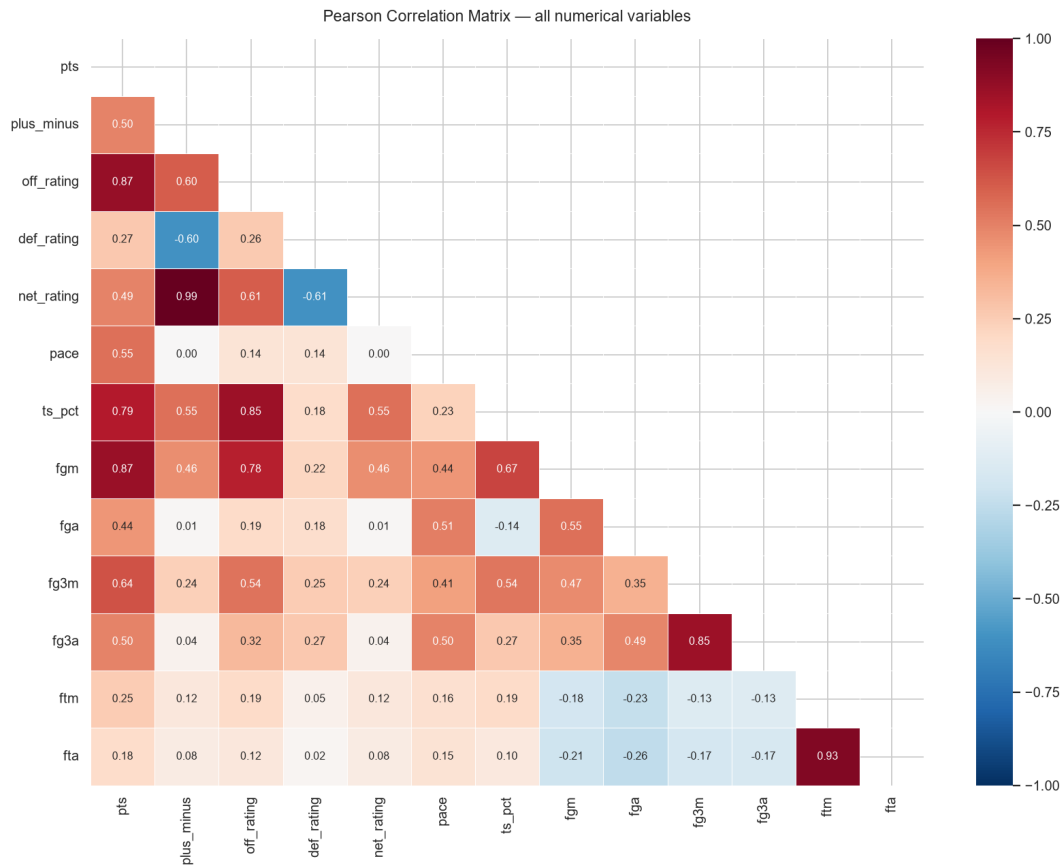


Figura 2.8: Matrice di correlazione tra feature numeriche e target principali.

La Figura 2.8 mostra dipendenze tra efficienza, ritmo, differenziali e risultati. Le correlazioni non sono tali da giustificare un modello puramente lineare: servono algoritmi capaci di combinare segnali deboli e interazioni.

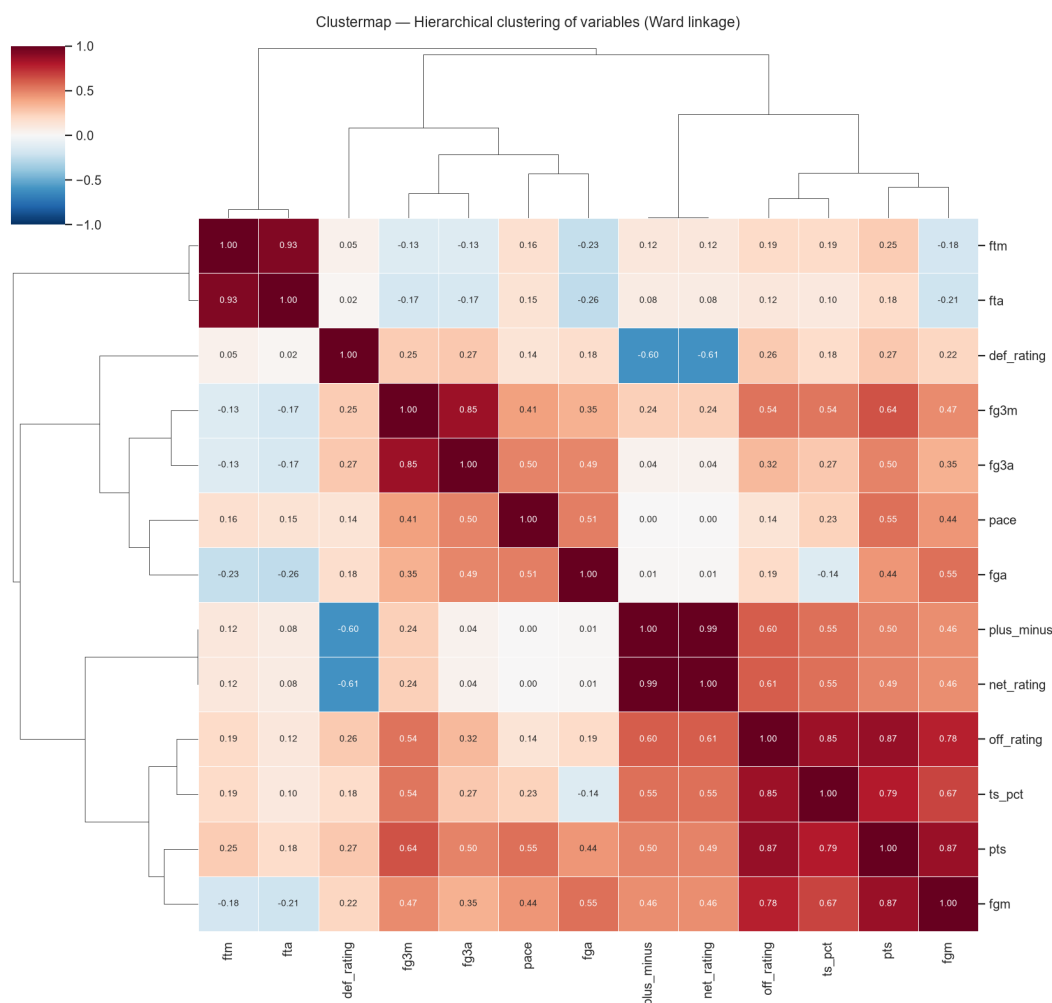


Figura 2.9: Clustermap con dendrogramma delle correlazioni tra variabili numeriche.

La Figura 2.9 usa un dendrogramma, cioè una rappresentazione gerarchica che raggruppa le variabili con profili di correlazione simili: rami più vicini indicano feature più correlate tra loro. Le associazioni più forti sono tra `plus_minus` e `net_rating` (0.99), tra `ftm` e `fta` (0.93), tra `off_rating` e `pts` (0.87), tra `pts` e `fgm` (0.87), tra `fg3m` e `fg3a` (0.85) e tra `off_rating` e `ts_pct` (0.85). `def_rating` mostra invece correlazioni negative con `net_rating` e `plus_minus`.

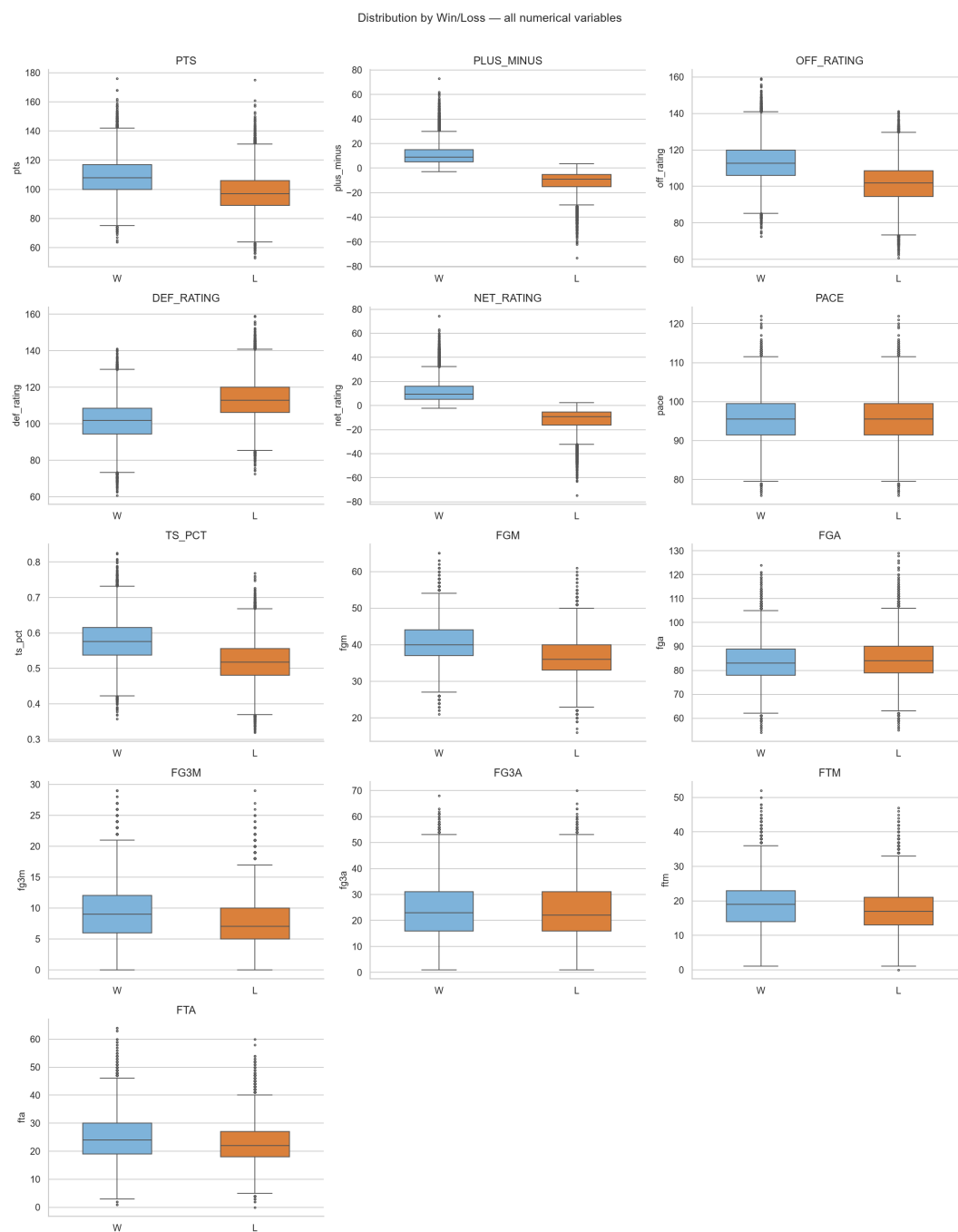


Figura 2.10: Scatter plot tra differenziali tecnici e differenziale punti finale.

La Figura 2.10 collega le feature differenziali al margine reale. La relazione è presente ma dispersa: la regressione può stimare una tendenza media, non determinare esattamente ogni partita.

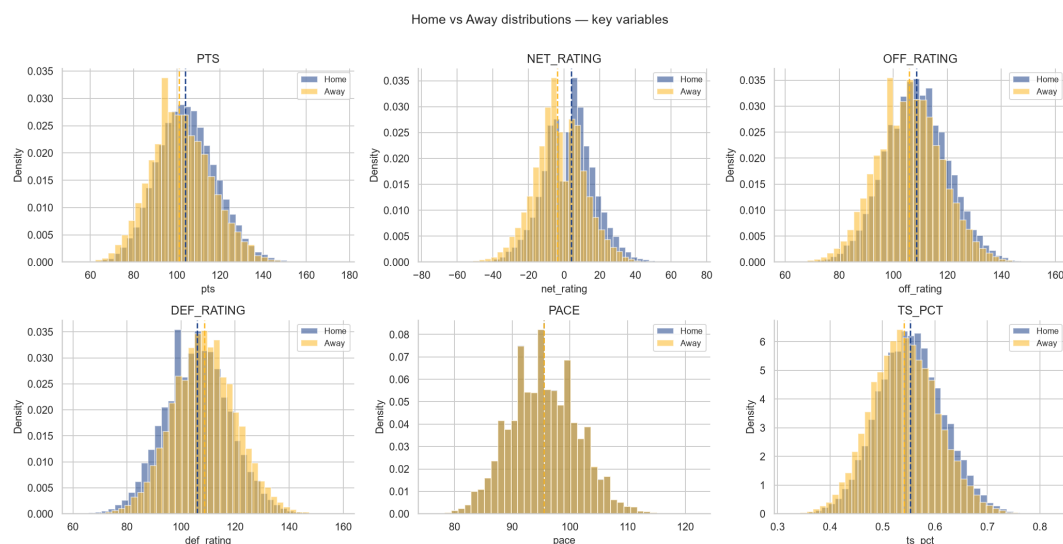


Figura 2.11: Confronto home-away delle statistiche di efficienza e ritmo.

La Figura 2.11 confronta le distribuzioni delle principali variabili prestazionali tra osservazioni Home e Away. Le linee tratteggiate indicano i valori medi: per `pts`, `net_rating`, `off_rating` e `ts_pct` la distribuzione home risulta leggermente spostata verso valori più alti, mentre per `def_rating` lo spostamento è nella direzione opposta. `PACE` mostra invece una sovrapposizione quasi completa tra casa e trasferta.

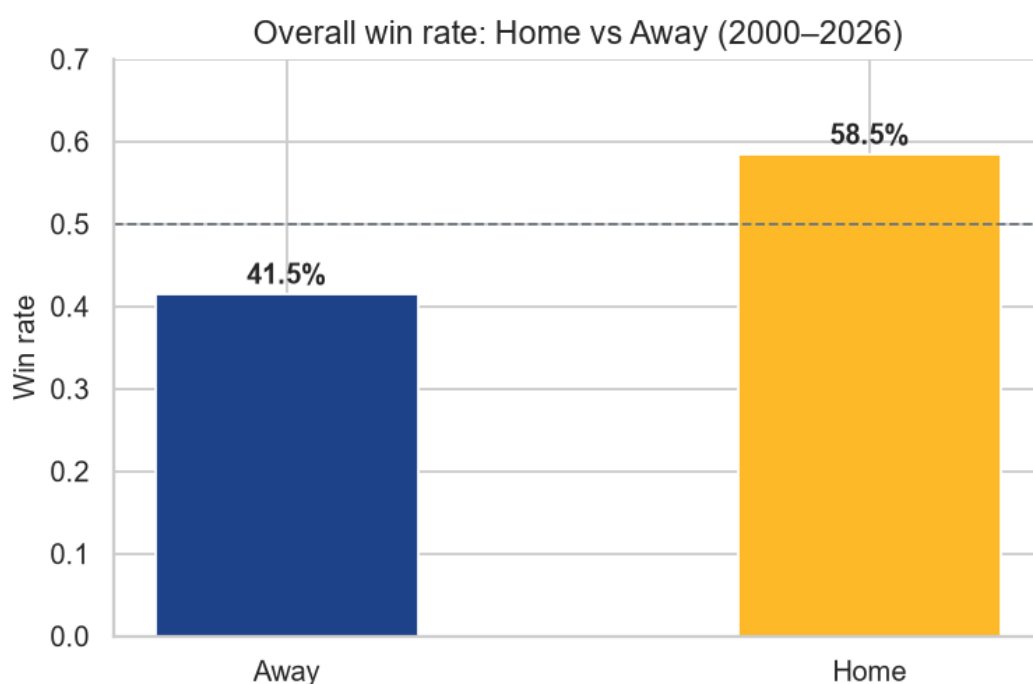


Figura 2.12: Win rate complessivo home-away nel periodo 2000–2026.

La Figura 2.12 mostra il win rate complessivo distinto tra casa e trasferta nel periodo 2000–2026. La squadra di casa vince il 58.5% delle osservazioni, mentre la squadra in trasferta vince il 41.5%; la linea tratteggiata al 50% consente di visualizzare lo scostamento rispetto a una distribuzione perfettamente bilanciata.

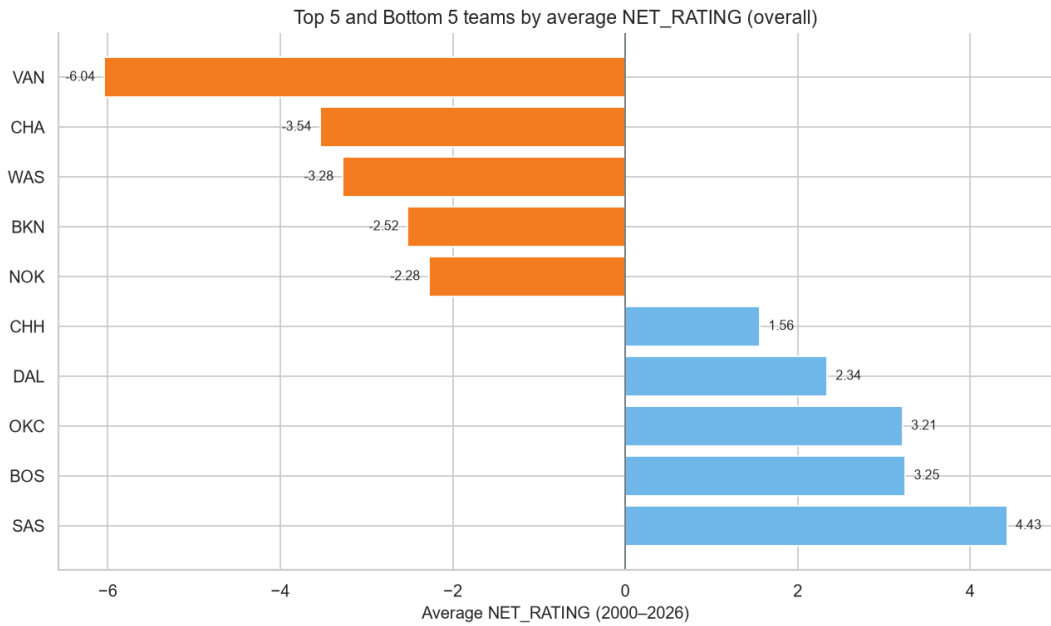


Figura 2.13: Trend stagionali delle relazioni tra prestazione e target.

La Figura 2.13 mostra che le relazioni statistiche non sono completamente stazionarie. Questo rafforza la scelta di testare i modelli su stagioni future e non su campioni mescolati.

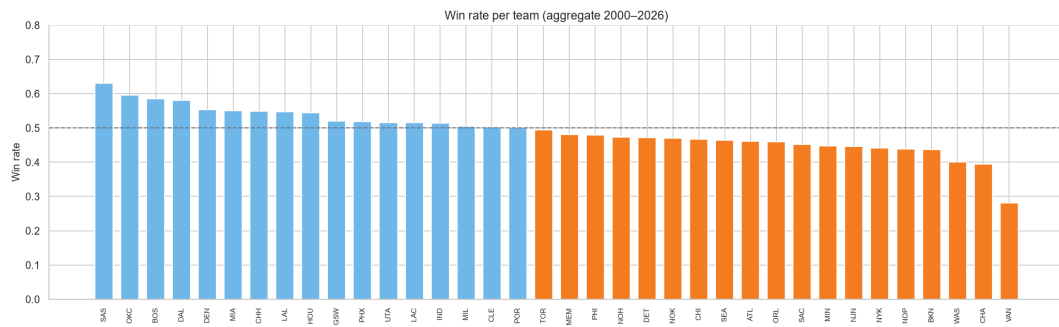


Figura 2.14: Sintesi visuale delle interazioni più rilevanti tra feature predittive.

La Figura 2.14 conferma che il dataset contiene interazioni multiple tra forma, efficienza e contesto. L'impatto sul modello è diretto: alberi e boosting sono adeguati per partizionare lo spazio in regioni decisionali non lineari.

2.7 Discussione critica

I grafici dell'EDA mostrano prima di tutto che il dataset rispetta due controlli di coerenza fondamentali. Le osservazioni W e L sono bilanciate al 50% e 50%, come atteso quando ogni partita genera una riga per la squadra vincente e una per la squadra perdente. Anche le osservazioni Home e Away risultano bilanciate al 50% e 50%, confermando che ogni gara è rappresentata simmetricamente dal punto di vista casa/trasferta.

Le distribuzioni delle variabili prestazionali sono concentrate in intervalli coerenti con statistiche NBA, ma non sono prive di code. I boxplot mettono in evidenza osservazioni estreme per punti, rating, pace e TS_PCT; il differenziale punti mostra una dispersione

ampia, quindi la previsione del margine è più difficile della sola classificazione dell'esito. Questi grafici indicano che gli outlier fanno parte della struttura osservata e devono essere considerati nella valutazione dei modelli.

La clustermap con dendrogramma evidenzia gruppi di variabili molto correlate. Le coppie più forti sono `plus_minus/net_rating`, `ftm/fta`, `off_rating/pts`, `pts/fgm`, `fg3m/fg3a` e `off_rating/ts_pct`. Questo risultato segnala ridondanza informativa tra alcune statistiche tecniche e giustifica l'attenzione alle feature derivate e ai differenziali nella fase successiva.

Il confronto home-away mostra infine un effetto casa misurabile: il win rate delle osservazioni Home è 58.5%, mentre quello delle osservazioni Away è 41.5%. Le distribuzioni di `pts`, `net_rating`, `off_rating` e `TS_PCT` sono leggermente spostate verso valori più alti per la squadra di casa; PACE, invece, risulta quasi sovrapposto tra casa e trasferta.

Nel complesso, l'EDA conferma che il dataset è coerente nei controlli di base, contiene segnali tecnici correlati tra loro e presenta un vantaggio casa osservabile. Le conclusioni restano descrittive: i grafici non dimostrano da soli la bontà dei modelli, ma indicano quali strutture statistiche devono essere rappresentate nella fase di feature engineering e validate nel capitolo di modellazione.

Capitolo 3

Feature Engineering e Modellazione

3.1 Sommario

Il feature engineering trasforma eventi storici in conoscenza temporale utilizzabile. In un dominio sportivo, una riga partita non basta: occorre rappresentare lo stato dinamico delle squadre prima della gara.

3.2 Fondamenti teorici del Feature Engineering

Il Feature Engineering trasforma dati grezzi in variabili predittive. Nel progetto non aggiunge semplici colonne descrittive, ma costruisce una rappresentazione pre-partita dello stato delle squadre: forma recente, efficienza, ritmo, riposo e vantaggio relativo rispetto all'avversario.

Il vincolo metodologico centrale è evitare leakage. Una feature usata per predire una partita al tempo t deve essere calcolata solo con informazioni precedenti a t . Le rolling windows formalizzano questa idea:

$$\bar{x}_{t,k} = \frac{1}{k} \sum_{i=t-k}^{t-1} x_i$$

La scelta della finestra k bilancia stabilità e reattività: finestre corte seguono rapidamente la forma recente, finestre lunghe riducono rumore. Poiché una partita NBA è un confronto, molte feature vengono poi trasformate in differenziali:

$$\Delta f_t = f_{home,t} - f_{away,t}$$

Un valore positivo indica vantaggio della squadra di casa sulla dimensione osservata; un valore negativo segnala vantaggio della squadra ospite. Questa impostazione consente al modello di apprendere differenze relative invece di livelli assoluti isolati.

3.3 Modellazione del problema

Ogni osservazione finale rappresenta una partita prima del suo svolgimento. La riga contiene lo stato recente della squadra di casa, lo stato recente della squadra ospite e il contesto temporale in cui l'incontro avviene. Il problema non è quindi descrivere una squadra in astratto, ma stimare l'esito di un confronto.

Il target `home_win` formalizza il task di classificazione: stabilire se la squadra di casa vincerà. Il target `point_differential` formalizza il task di regressione: stimare il margine finale casa-ospite. La doppia formulazione è utile perché una decisione binaria e una stima del margine rispondono a domande diverse: la prima indica l'esito, la seconda misura intensità e confidenza.

3.4 Implementazione

L'implementazione opera su dati ordinati cronologicamente. Le statistiche rolling sono calcolate separatamente per squadra, usando solo partite precedenti; successivamente le righe home e away vengono unite a livello partita. Le feature di calendario derivano dalle date delle gare precedenti, mentre gli z-score normalizzano alcune misure rispetto alla distribuzione della stagione. Il risultato è un dataset a livello partita usato dal modulo ML e, in forma ridotta, anche dal CSP per individuare big match.

3.5 Strumenti utilizzati

La pipeline usa rolling windows ordinate temporalmente, join home-away, indicatori di calendario e trasformazioni statistiche. Le feature sono organizzate in famiglie: identificativi e datazione della partita; statistiche recenti della squadra di casa; statistiche recenti della squadra ospite; indicatori di riposo e back-to-back; z-score stagionali; differenziali home-away; target di classificazione e regressione. Questa organizzazione evita un semplice elenco tecnico e chiarisce il ruolo conoscitivo di ogni gruppo di variabili.

3.6 Decisioni di progetto

La rolling win rate su k partite è definita come:

$$WR_t^k = \frac{1}{k} \sum_{i=t-k}^{t-1} win_i$$

Il net rating rolling è:

$$NR_t^k = \frac{1}{k} \sum_{i=t-k}^{t-1} (OffRtg_i - DefRtg_i)$$

Il True Shooting Percentage è:

$$TS\% = \frac{PTS}{2(FGA + 0.44 \cdot FTA)}$$

I differenziali home-away sono ottenuti come $\Delta x = x_{home} - x_{away}$, scelta che rende esplicita la comparazione tra le due squadre nel momento della partita.

Oltre a queste feature principali, il progetto introduce ulteriori variabili ingegnerizzate:

- **Rest days:** numero di giorni trascorsi dall'ultima partita della squadra. Serve a rappresentare recupero fisico e fatica.

- **Back-to-back:** indicatore binario pari a 1 quando una squadra gioca senza adeguato riposo. È una feature direttamente collegata allo scheduling.
- **Streak:** sequenza corrente di vittorie o sconfitte. Sintetizza inerzia competitiva e forma psicologica.
- **Season z-score:** normalizzazione rispetto alla distribuzione della stagione, utile per confrontare squadre in contesti storici diversi.
- **Win rate precedente:** misura sintetica della qualità storica della squadra nella stagione precedente.
- **Pace differential:** differenza tra il ritmo recente delle due squadre, utile per rappresentare possibili mismatch di velocità di gioco.
- **TS% z-score differential:** confronto normalizzato dell'efficienza di tiro rispetto alla stagione, risultato centrale anche nelle feature importance.

Le restanti feature possono essere raggruppate in: identificativi di partita e stagione; statistiche di box score; percentuali di tiro; medie rolling di punti, pace e TS%; indicatori home/away; target `home_win` e `point_differential`.

3.7 Risultati

Il dataset feature engineered contiene 58 colonne e 31,160 partite. Il risultato principale del capitolo è la trasformazione di record squadra-partita in esempi pre-partita adatti al Machine Learning. Ogni riga finale contiene informazione storica disponibile prima della gara, il confronto tra squadra di casa e ospite e i target sperimentali.

La presenza di feature rolling consente di rappresentare la forma recente; le feature di calendario aggiungono una dimensione logistica; i differenziali home-away rendono esplicito il confronto competitivo. Questa struttura è coerente con il vincolo anti-leakage e permette di usare lo stesso dataset sia per classificazione sia per regressione.

3.8 Valutazione e discussione critica

L'elemento più importante è la temporalità. Una feature calcolata includendo la partita corrente produrrebbe leakage e metriche artificialmente alte. La pipeline evita questo problema usando rolling precedenti e split temporali.

Il limite principale è che le feature descrivono prestazioni aggregate, ma non includono informazione su infortuni, rotazioni, disponibilità dei giocatori o mercato delle quote. Inoltre, la scelta della finestra rolling è un compromesso: una finestra fissa non sempre cattura allo stesso modo squadre stabili e squadre in rapida trasformazione. Un miglioramento futuro potrebbe confrontare finestre multiple o feature pesate esponenzialmente.

Nel complesso, il feature engineering fornisce una rappresentazione solida e interpretabile del dominio NBA, ma non esaurisce tutte le fonti informative che influenzano una partita reale.

Capitolo 4

Addestramento e Validazione del Modello

4.1 Sommario

Il modulo di apprendimento supervisionato rappresenta la componente predittiva del KBS. A partire dal dataset `features_nba_data_2000-01_2025-26.csv`, il sistema affronta due problemi distinti ma collegati: la classificazione della vittoria della squadra di casa (`home_win`) e la regressione del differenziale punti (`point_differential`). La classificazione produce una decisione binaria direttamente utilizzabile in scenari di previsione dell'esito; la regressione fornisce invece una stima più informativa dell'intensità del vantaggio, utile per interpretare il margine atteso e costruire indicatori di confidenza.

4.2 Fondamenti teorici del Machine Learning supervisionato

L'apprendimento supervisionato studia il problema di apprendere una funzione a partire da esempi etichettati. Dato un insieme di osservazioni $\{(x_i, y_i)\}_{i=1}^n$, dove x_i è un vettore di feature e y_i è il target osservato, l'obiettivo è stimare una funzione $\hat{f}$ tale che:

$$\hat{y} = \hat{f}(x)$$

produca predizioni accurate su nuovi esempi non osservati. Nel progetto NBA, x_i contiene feature di forma, efficienza e calendario; y_i è la vittoria della squadra di casa per la classificazione o il differenziale punti per la regressione.

La classificazione riguarda target discreti. Nel caso di `home_win`, il modello stima se l'esito della partita corrisponde a una vittoria della squadra di casa oppure no. La regressione riguarda invece target continui: il differenziale punti può assumere valori positivi o negativi e misura l'intensità del risultato. I due task sono collegati ma non equivalenti: una vittoria di 1 punto e una vittoria di 30 punti hanno la stessa etichetta classificatoria ma significato regressivo molto diverso.

Ogni modello viene addestrato minimizzando una funzione di perdita. Per la classificazione, una perdita comune è la log loss:

$$L(y, p) = -y \log(p) - (1 - y) \log(1 - p)$$

dove p è la probabilità stimata della classe positiva. Per la regressione, sono frequenti perdite basate sull'errore quadratico o assoluto:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

La scelta della metrica influenza l'interpretazione: RMSE penalizza molto gli errori estremi, mentre MAE descrive l'errore medio in punti in modo più robusto.

Il concetto centrale è la generalizzazione. Un modello non deve soltanto adattarsi ai dati di training, ma mantenere performance su stagioni future.

Il bias-variance tradeoff descrive il compromesso tra errore sistematico e sensibilità ai dati. Modelli ad alto bias semplificano eccessivamente il problema; modelli ad alta varianza cambiano molto al variare del campione. Gli ensemble usati nel progetto rispondono proprio a questo problema: Random Forest riduce la varianza aggregando alberi diversi, mentre Gradient Boosting e XGBoost riducono progressivamente il bias costruendo modelli additivi.

Nei dati temporali la validazione è parte della teoria, non un dettaglio sperimentale. Se i dati hanno ordine cronologico, mescolare passato e futuro può introdurre leakage. Un modello valutato casualmente potrebbe sfruttare distribuzioni future indirettamente presenti nel training set. Per questo motivo il progetto usa split temporali e walk-forward validation: il modello viene valutato in condizioni più simili all'uso reale.

4.3 Modellazione del problema NBA

Nel dominio NBA, l'apprendimento supervisionato viene formulato come predizione pre-partita. Le feature rappresentano informazione disponibile prima dell'incontro: statistiche rolling, riposo, back-to-back, forma recente e differenziali home-away. La classificazione stima la probabilità di vittoria della squadra di casa; la regressione stima il margine atteso. Questa doppia modellazione permette di distinguere decisione e intensità: sapere chi vincerà e stimare di quanto sono informazioni complementari.

4.4 Ensemble Learning

Nel progetto vengono confrontati tre modelli ad alberi, scelti perché adatti a dati tabellari con interazioni tra forma recente, efficienza, differenziali home-away e calendario. Le configurazioni sono quelle definite nei moduli di costruzione dei modelli.

Tabella 4.1: Configurazione del modello Random Forest.

Parametro	Valore	Descrizione
<code>n_estimators</code>	300	Numero di alberi usati per stabilizzare le predizioni senza aumentare eccessivamente il costo computazionale.
<code>max_depth</code>	None	Alberi lasciati crescere senza profondità massima esplicita; la regolarizzazione è affidata alle foglie.
<code>min_samples_leaf</code>	5	Numero minimo di campioni per foglia; riduce la varianza su dati NBA rumorosi.
<code>class_weight</code>	balanced	Usato solo nella classificazione per compensare la prior home-win.
<code>n_jobs</code>	-1	Usa tutti i core disponibili durante l'addestramento.
<code>random_state</code>	42	Rende riproducibile la costruzione degli alberi.

Tabella 4.2: Configurazione del modello Gradient Boosting.

Parametro	Valore	Descrizione
<code>n_estimators</code>	300	Numero di stadi di boosting, scelto per bilanciare capacità predittiva e costo.
<code>learning_rate</code>	0.05	Passo conservativo che riduce il contributo di ogni albero.
<code>max_depth</code>	4	Profondità contenuta, adatta a combinare segnali NBA deboli senza produrre alberi troppo complessi.
<code>subsample</code>	0.8	Usa l'80% delle righe a ogni stadio, introducendo stochastic gradient boosting.
<code>random_state</code>	42	Rende riproducibile la procedura di addestramento.

Tabella 4.3: Configurazione del modello XGBoost.

Parametro	Valore	Descrizione
<code>n_estimators</code>	500	Limite massimo di alberi; l'early stopping può interrompere prima l'addestramento.
<code>learning_rate</code>	0.05	Passo conservativo per migliorare la generalizzazione.
<code>max_depth</code>	6	Profondità standard per dati tabellari, utile per catturare interazioni tra feature.
<code>subsample</code>	0.8	Campionamento delle righe per ogni albero, usato per ridurre overfitting.
<code>colsample_bytree</code>	0.8	Campionamento delle colonne per ogni albero, utile con molte feature ingegnerizzate.
<code>early_stopping_rounds</code>	50	Ferma l'addestramento se la metrica di validazione non migliora per 50 round.
<code>objective</code>	<code>binary:logistic</code> / <code>reg:squarederror</code>	Obiettivo usato rispettivamente per classificazione e regressione.
<code>eval_metric</code>	<code>auc</code> / <code>rmse</code>	Metrica di validazione usata rispettivamente per classificazione e regressione.
<code>n_jobs</code>	-1	Usa tutti i core disponibili durante l'addestramento.
<code>random_state</code>	42	Rende riproducibile l'addestramento.
<code>verbosity</code>	0	Disattiva output non necessario durante il training.

4.5 Strumenti utilizzati

La pipeline utilizza Scikit-learn per Random Forest e Gradient Boosting, XGBoost per il modello di boosting regolarizzato, Pandas e NumPy per la gestione tabellare, Joblib per la persistenza dei modelli e Matplotlib/Seaborn per la diagnostica.

4.6 Pipeline e decisioni di progetto

Tabella 4.4: Split temporale adottato nella pipeline ML.

Split	Stagioni	Partite
Train	2000-01–2018-19	22 881
Validation	2019-20–2021-22	3 369
Test	2022-23–2025-26	4 910

La decisione più importante è lo split temporale. Nel dominio sportivo, un random split consentirebbe al modello di vedere indirettamente distribuzioni e dinamiche di stagioni future, generando una valutazione troppo ottimistica. Il progetto separa quindi training, validation e test per stagioni: il modello apprende dal passato e viene valutato su stagioni successive, simulando l'uso reale di un sistema predittivo prima dell'inizio o durante una stagione NBA.

4.6.1 Baseline

La baseline è il riferimento minimo contro cui valutare i modelli. Per la classificazione, la baseline maggioritaria predice sempre la classe più frequente nel training set, cioè la vittoria della squadra di casa. Nel progetto la baseline di training è 0.5967; applicata al validation set vale 0.5462 e sul test set 0.5556. Un modello è utile solo se supera in modo netto questa regola semplice e riproducibile.

Per la regressione, la baseline predice il valore medio del differenziale punti appreso sul training set. Le metriche baseline riportate nei notebook sono MAE validation pari a 11.8563 e MAE test pari a 12.2437. I modelli regressivi devono quindi ridurre l'errore medio rispetto a una previsione costante. Questa doppia baseline chiarisce che le performance sperimentali non vengono valutate in astratto, ma rispetto a regole semplici e riproducibili.

4.6.2 Walk-Forward Cross Validation

Il random split non è appropriato quando l'obiettivo è simulare predizioni future. Nei dati NBA, partite della stessa stagione condividono contesto tecnico, roster, stile di gioco e condizioni di calendario. Se partite future entrano nel training set, il modello può beneficiare di informazione non disponibile al momento della predizione reale. Questo è un caso di leakage temporale.

La Walk-Forward Cross Validation risolve il problema imponendo un ordine causale: si addestra su stagioni precedenti e si valida su una stagione successiva. Formalmente, per fold k si usa un insieme di training $T_k = \{s_1, \dots, s_k\}$ e una stagione di validazione s_{k+1} . Nel notebook di confronto dei modelli i fold interni coprono le stagioni 2014–15–2018–19: a ogni iterazione il preprocessing viene rifittato solo sul training del fold e poi applicato alla stagione successiva, simulando l'arrivo di dati futuri non osservati.

Questo schema misura sia performance sia stabilità. Nei dati sportivi è particolarmente importante perché il gioco evolve: aumentano tiri da tre, cambia il pace, cambiano strategie e distribuzioni statistiche. La presenza della stagione 2019–20 nel blocco di validation ufficiale e la sua esclusione dai fold interni riducono inoltre il rischio di confondere robustezza generale con adattamento a una stagione eccezionale.

Nel confronto dei modelli, le metriche dei fold vengono sintetizzate tramite media μ e deviazione standard σ :

$$\mu = \frac{1}{K} \sum_{k=1}^K m_k, \quad \sigma = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (m_k - \mu)^2}$$

dove m_k è la metrica osservata nel fold k . La media misura la performance attesa, mentre la deviazione standard misura stabilità. Un modello con μ alta ma σ elevata potrebbe dipendere troppo da specifiche stagioni; un modello con μ leggermente inferiore ma σ bassa può essere più robusto.

4.7 Risultati di validazione walk-forward

Tabella 4.5: Walk-forward cross-validation su cinque fold temporali interni al blocco di training.

Modello	Accuracy media	F1 medio	AUC-ROC medio
Random Forest	0.8488	0.8728	0.9247
XGBoost	0.8437	0.8677	0.9210
Gradient Boosting	0.8536	0.8766	0.9297

La walk-forward validation confronta i modelli su cinque fold temporali interni al blocco di training. Gradient Boosting ottiene i valori medi più alti su tutte le metriche riportate: accuracy 0.8536, F1 0.8766 e AUC-ROC 0.9297. Random Forest segue con accuracy 0.8488, F1 0.8728 e AUC-ROC 0.9247, mentre XGBoost registra accuracy 0.8437, F1 0.8677 e AUC-ROC 0.9210. I risultati indicano che tutti e tre i modelli superano ampiamente una classificazione casuale e che Gradient Boosting è il migliore nel confronto walk-forward medio, pur con distanze contenute rispetto agli altri ensemble.

4.8 Risultati di classificazione sul test set

Tabella 4.6: Confronto dei modelli di classificazione per il target `home_win`. Le metriche sono calcolate su validation e test set temporali.

Modello	Acc. Val	Acc. Test	F1 Val	F1 Test	AUC Val	AUC Test
Random Forest	0.8519	0.8525	0.8638	0.8653	0.9326	0.9291
XGBoost	0.8498	0.8548	0.8646	0.8707	0.9325	0.9302
Gradient Boosting	0.8480	0.8552	0.8632	0.8708	0.9320	0.9304

Sul test set finale, Gradient Boosting ottiene la migliore accuracy (0.8552), seguito da XGBoost (0.8548) e Random Forest (0.8525). I tre modelli sono molto vicini, ma tutti superano nettamente la baseline maggioritaria applicata al test set, pari a 0.5556. L'AUC-ROC intorno a 0.93 mostra che i modelli non stanno semplicemente scegliendo la classe più frequente: possiedono una buona capacità di ordinare le partite in base alla probabilità di vittoria della squadra di casa.

Tabella 4.7: Metriche derivate dalle matrici di confusione sul test set. La classe positiva è la vittoria della squadra di casa.

Modello	TN	FP	FN	TP	Precision Home	Recall Home
Random Forest	1860	322	402	2326	0.8784	0.8526
XGBoost	1797	385	328	2400	0.8618	0.8798
Gradient Boosting	1803	379	332	2396	0.8634	0.8783

La matrice di confusione chiarisce il profilo degli errori. Random Forest è il modello più prudente sulla vittoria della squadra di casa: produce meno falsi positivi e quindi

ottiene precision home più alta (0.8784), ma perde più vittorie reali della squadra di casa. XGBoost e Gradient Boosting hanno recall più elevata, intercettando più casi positivi ma accettando un numero maggiore di falsi positivi. Questa differenza ha impatto applicativo: se il sistema deve selezionare partite ad alta probabilità di successo della squadra di casa, Random Forest è più conservativo; se deve massimizzare il recupero dei casi positivi, Gradient Boosting e XGBoost sono preferibili.

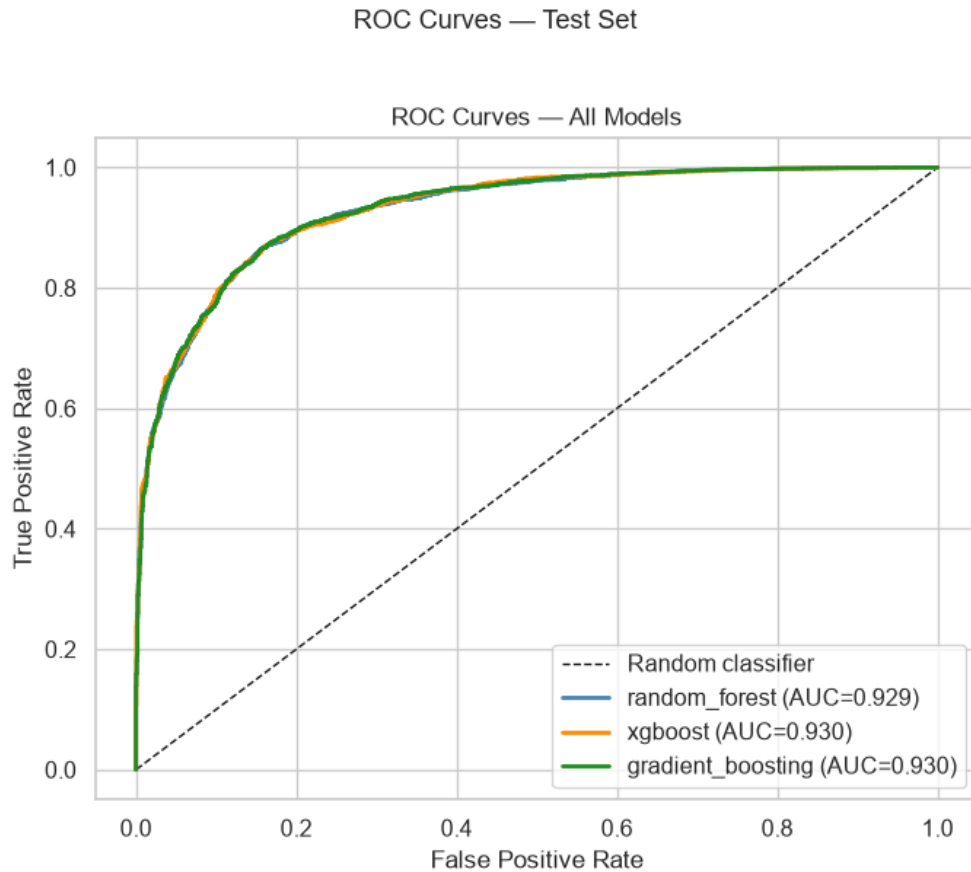


Figura 4.1: Curve ROC sul test set per Random Forest, XGBoost e Gradient Boosting.

La Figura 4.1 mostra curve molto vicine tra loro e nettamente superiori alla diagonale del classificatore casuale. Questo conferma che la probabilità stimata dai modelli contiene informazione discriminante reale. L'area sotto la curva è circa 0.93 per tutti gli algoritmi: la differenza tra i modelli non riguarda quindi la capacità generale di separare vittorie e sconfitte, ma il comportamento locale alla soglia decisionale e la robustezza sui casi ambigui.

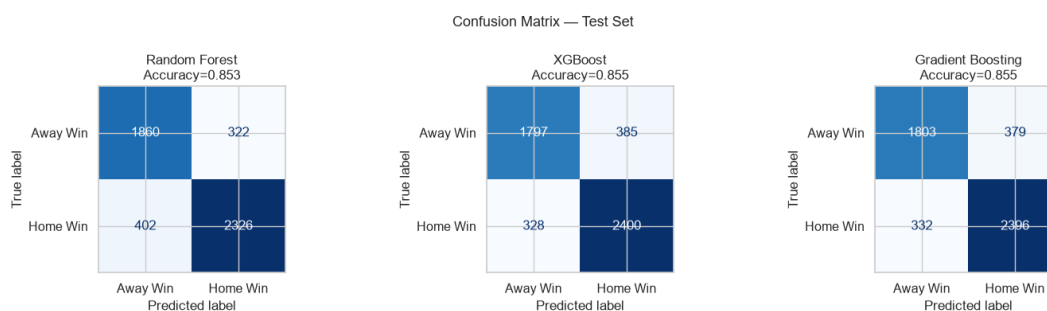


Figura 4.2: Matrici di confusione sul test set per il target `home_win`.

La Figura 4.2 rende visibile il compromesso tra falsi positivi e falsi negativi. Random Forest sbaglia meno quando predice `home_win=1`, ma lascia fuori più vittorie reali della squadra di casa. Gradient Boosting e XGBoost sono più aggressivi: recuperano più casi positivi e ottengono una migliore accuracy complessiva, ma accettano una precision leggermente più bassa.

4.9 Risultati di regressione

Tabella 4.8: Confronto dei modelli di regressione per il target `point_differential`. MAE e RMSE sono espressi in punti.

Modello	MAE Val	MAE Test	RMSE Val	RMSE Test	R^2 Val	R^2 Test
Random Forest	6.0664	6.5821	7.7292	8.3718	0.7309	0.7054
XGBoost	5.9828	6.4374	7.6238	8.2155	0.7382	0.7163
Gradient Boosting	5.9828	6.4609	7.6378	8.2334	0.7372	0.7151

La regressione sul differenziale punti è più difficile della classificazione perché il margine finale è sensibile a fattori non osservati: garbage time, rotazioni finali, infortuni in partita, falli intenzionali e gestione tattica degli ultimi possesi. Nonostante questo, XGBoost ottiene il miglior MAE test (6.4374 punti) e il miglior R^2 test (0.7163), seguito molto da vicino da Gradient Boosting. Il valore di R^2 indica che oltre il 70% della varianza del differenziale viene spiegata dalle feature disponibili, un risultato solido considerando la rumorosità del basket NBA.

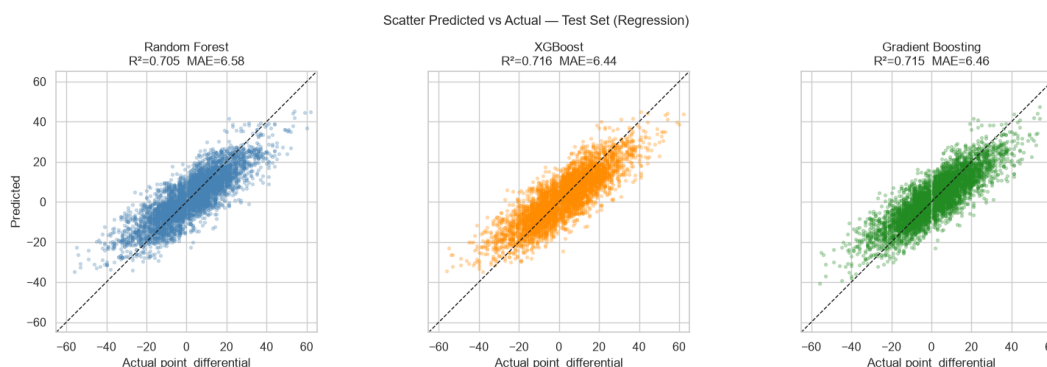


Figura 4.3: Confronto tra differenziale punti predetto e reale per i modelli di regressione.

La Figura 4.3 evidenzia una buona concentrazione dei punti attorno alla diagonale, ma anche una compressione verso il centro: i modelli tendono a sottostimare i margini estremi. Questo comportamento è atteso per modelli supervisionati addestrati a minimizzare errori medi: le partite con scarti molto ampi sono meno frequenti e spesso dipendono da fattori contingenti non presenti nei dati.

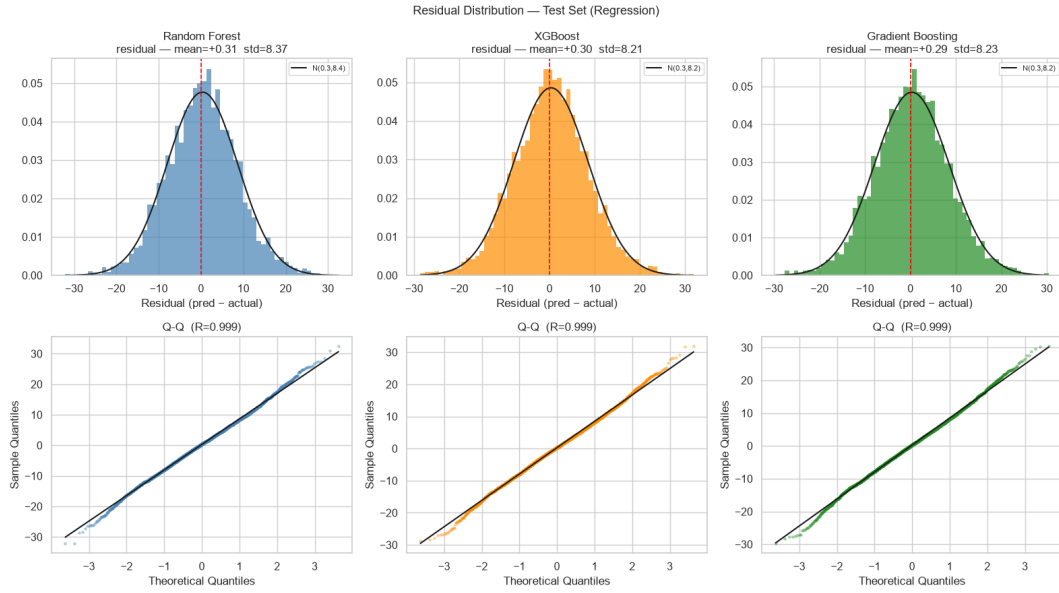


Figura 4.4: Distribuzione dei residui dei modelli di regressione sul test set.

La Figura 4.4 mostra residui centrati vicino allo zero, segnale di assenza di bias medio marcato. Le code, tuttavia, restano ampie: il modello non è progettato per prevedere perfettamente blowout o upset rari, ma per fornire una stima robusta nella maggioranza delle partite. Questo risultato giustifica l'uso del regressore come componente complementare e non come unico decisore.

4.10 Feature Importance e interpretabilità

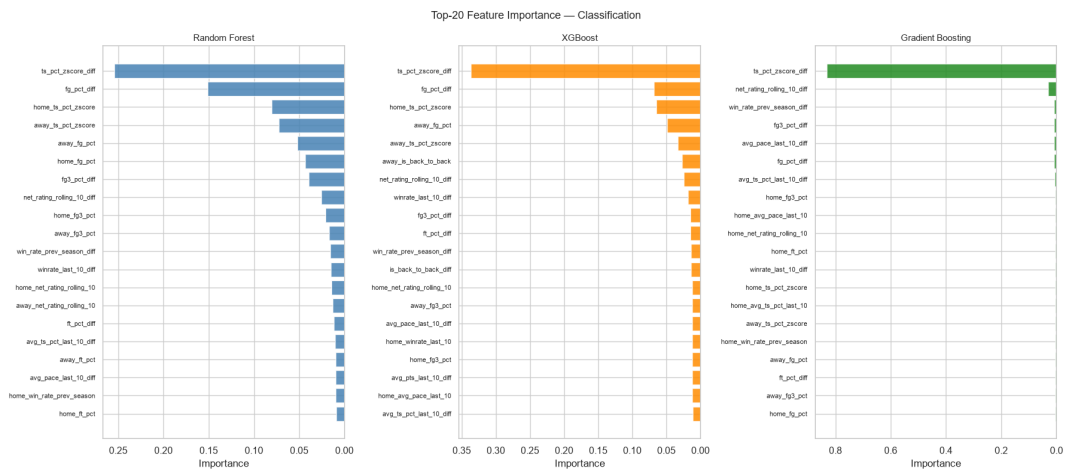


Figura 4.5: Top-20 Feature Importance per i modelli di classificazione.

La Figura 4.5 indica che le feature differenziali e rolling sono centrali nella previsione dell'esito. In particolare, il ranking globale identifica `ts_pct_zscore_diff` come predittore dominante. L'interpretazione sportiva è coerente: l'efficienza di tiro recente, confrontata tra casa e trasferta, riassume qualità offensiva, selezione dei tiri e forma della squadra.

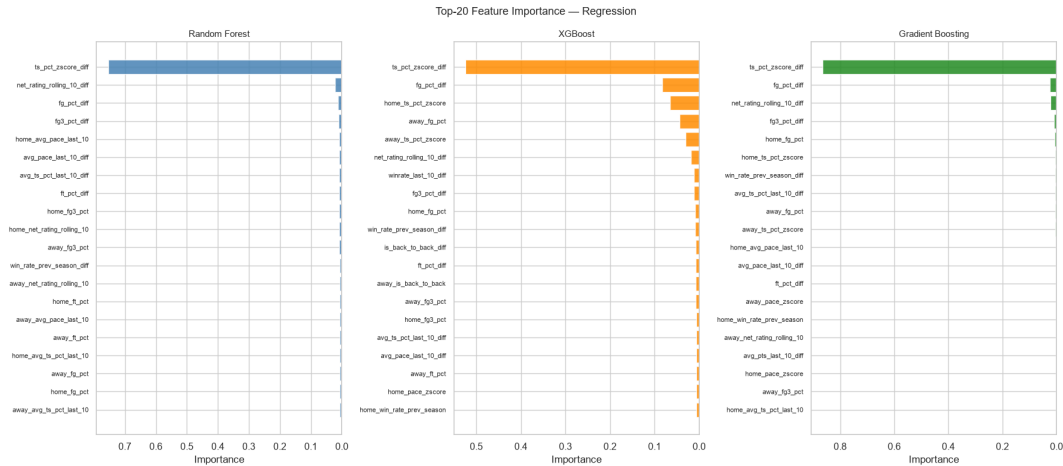


Figura 4.6: Top-20 Feature Importance per i modelli di regressione del differenziale punti.

La Figura 4.6 conferma che anche la stima del margine dipende soprattutto da differenziali di efficienza e forma recente. La convergenza tra classificazione e regressione è un segnale positivo: i due task non stanno apprendendo pattern contraddittori, ma due letture diverse della stessa conoscenza cestistica.

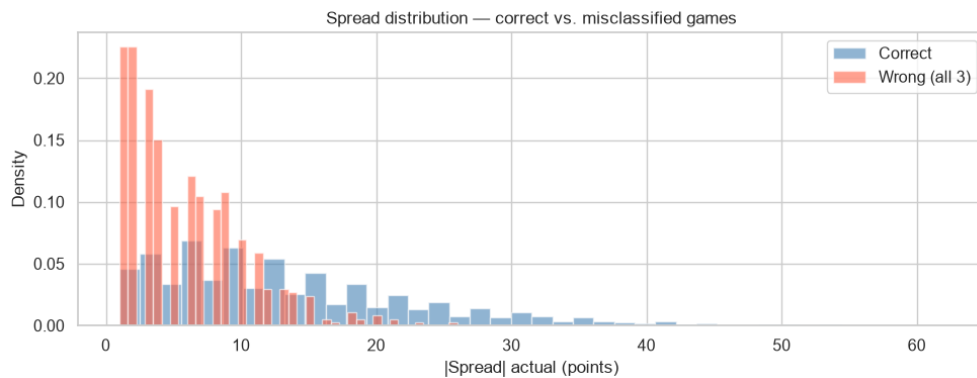


Figura 4.7: Confronto tra accuratezza derivata dal regressore e classificazione diretta.

La Figura 4.7 confronta l'uso del segno del differenziale predetto con la classificazione diretta. Le differenze sono minime: Random Forest e XGBoost ottengono risultati quasi equivalenti, mentre Gradient Boosting conserva un piccolo vantaggio nella classificazione diretta. Questo suggerisce una strategia integrata: usare il classificatore per la decisione binaria e il regressore per spiegare intensità e confidenza.

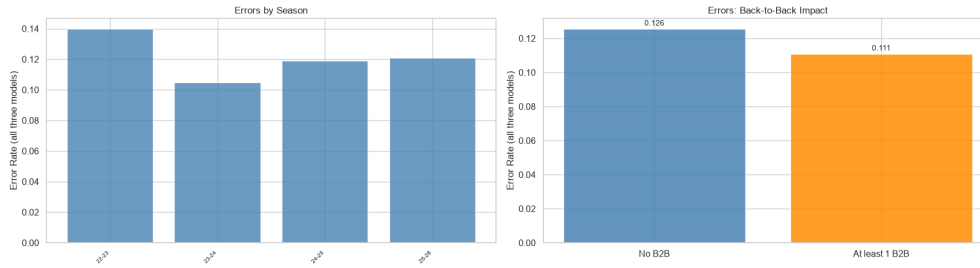


Figura 4.8: Analisi degli errori per stagione e condizioni back-to-back.

La Figura 4.8 sposta la valutazione dal valore medio alle condizioni di fallimento. L'analisi riporta 595 partite errate da tutti e tre i modelli, pari al 12.1% del test set.

4.11 Scelta finale del modello

Per il task primario `home_win`, XGBoost è una scelta prudente come classificatore operativo: pur avendo un'accuracy test di poco inferiore a Gradient Boosting (0.8548 contro 0.8552), combina prestazioni quasi equivalenti, AUC elevata e buona stabilità complessiva. Gradient Boosting resta il modello più stabile nella Walk-Forward Cross Validation e il migliore per accuracy test, quindi rappresenta un'alternativa pienamente competitiva.

Per il task `point_differential`, XGBoost è il regressore preferibile per MAE, RMSE e R^2 sul test set. La decisione non è puramente numerica: il classificatore serve per la decisione binaria, mentre il regressore fornisce una misura complementare di margine e confidenza.

Capitolo 5

Modellazione CSP

5.1 Sommario

Il modulo CSP/COP affronta la pianificazione televisiva di partite NBA come problema di assegnazione con vincoli. Nel dominio reale, una partita non deve essere soltanto collocata in un orario disponibile: deve rispettare vincoli di calendario, non sovrapporsi in modo dannoso con altre partite rilevanti, valorizzare i big match e limitare gli effetti negativi su riposo e trasferte. La modellazione con vincoli rende esplicita questa conoscenza e consente di separare regole inderogabili, regole commerciali e funzione obiettivo.

5.2 Knowledge Base

La Knowledge Base del modulo è costituita da tre insiemi principali: squadre, partite e slot televisivi. Le partite sono estratte dal dataset feature engineered `features_nba_data_2000-01_2025-26.csv` e riguardano il periodo 13–15 dicembre 2022. Il criterio di big match non è arbitrario: una partita è marcata come rilevante quando entrambe le squadre hanno una win rate rolling sulle ultime dieci partite almeno pari a 0.6. In questo modo il CSP riusa conoscenza prodotta dal feature engineering e mantiene continuità con il modulo ML.

Tabella 5.1: Dimensione dell’istanza CSP/COP costruita su partite NBA reali del 13–15 dicembre 2022.

Grandezza	Valore
Partite selezionate	13
Big match identificati	4
Slot televisivi disponibili	14
Slot prime time	8
Variabili booleane $x_{m,s}$	182
Vincoli totali	118
Vincoli di assegnazione	13
Vincoli di capacità slot	14
Vincoli team-giorno	76
Vincoli business	15
Stato solver	OPTIMAL
Valore obiettivo	4

L'istanza è compatta ma rappresentativa: 13 partite reali, 14 slot, 4 big match e 8 slot prime time. La scelta di una scala ridotta è didatticamente utile per verificare la soluzione, ma la formulazione è scalabile perché il numero di variabili cresce come $|M| \cdot |S|$ e ogni famiglia di vincoli è definita in forma parametrica.

5.2.1 Rappresentazione della conoscenza

La rappresentazione scelta è ibrida: dichiarativa nella definizione di fatti e vincoli, procedurale nell'attivazione del solver. I fatti descrivono oggetti del dominio: una squadra ha un identificativo, una partita collega due squadre e uno slot possiede giorno, orario, canale e attributo prime time. I vincoli traducono policy e regole operative. La conoscenza non viene dunque incorporata in condizioni sparse nel codice, ma organizzata come modello matematico interrogabile.

Se cambia il criterio di big match, si modifica la regola che costruisce M^B ; se cambia la disponibilità televisiva, si modifica l'insieme S ; se la lega introduce una regola di riposo, si aggiunge una famiglia di vincoli. La struttura del ragionamento resta invariata. Il modello è quindi manutenibile, estendibile e spiegabile.

5.3 Modellazione CSP

Sia M l'insieme delle partite e S l'insieme degli slot. Per ogni coppia (m, s) si introduce la variabile:

$$x_{m,s} = \begin{cases} 1 & \text{se la partita } m \text{ è assegnata allo slot } s, \\ 0 & \text{altrimenti.} \end{cases}$$

La rappresentazione binaria è naturale: ogni variabile risponde a una domanda atomica della KB, cioè se una specifica partita occupi uno specifico slot. Il dominio è quindi discreto, finito e interpretabile.

5.3.1 Vincoli hard

I vincoli hard definiscono la fattibilità minima. Ogni partita deve essere trasmessa una sola volta:

$$\sum_{s \in S} x_{m,s} = 1 \quad \forall m \in M$$

Ogni slot può contenere al massimo una partita:

$$\sum_{m \in M} x_{m,s} \leq 1 \quad \forall s \in S$$

Per evitare conflitti sportivi, una squadra non può comparire in più partite nello stesso giorno:

$$\sum_{m \in M_t} \sum_{s \in S_d} x_{m,s} \leq 1 \quad \forall t, \forall d$$

dove M_t è l'insieme delle partite che coinvolgono la squadra t e S_d è l'insieme degli slot nel giorno d .

5.3.2 Vincoli business

I vincoli business modellano il valore televisivo. Il prime time è una risorsa limitata; non tutte le partite possono essere spinte nelle finestre più pregiate. Si introduce quindi un cap:

$$\sum_{m \in M} \sum_{s \in S^P} x_{m,s} \leq N$$

dove S^P è l'insieme degli slot prime time. Per i big match si evita inoltre la sovrapposizione nello stesso canale e nella stessa finestra, per non ridurre reciprocamente audience e valore commerciale.

La funzione obiettivo massimizza l'esposizione dei big match in prime time:

$$\max \sum_{m \in M^B} \sum_{s \in S^P} x_{m,s}$$

5.3.3 Formalizzazione matematica

La formulazione compatta del problema è:

$$\max_x f(x) \quad \text{s.t.} \quad x_{m,s} \in \{0, 1\}, \mathcal{H}, \mathcal{B}$$

dove $x_{m,s}$ indica l'assegnazione della partita m allo slot s , $\mathcal{H}$ è l'insieme dei vincoli hard e $\mathcal{B}$ l'insieme dei vincoli business. La funzione obiettivo è:

$$f(x) = \sum_{m \in M^B} \sum_{s \in S^P} x_{m,s}$$

cioè il numero di big match collocati in prime time. Il solver restituisce una soluzione ottima con valore obiettivo 4: tutti e quattro i big match selezionati vengono collocati in prime time rispettando i vincoli. Questo risultato è importante perché mostra che l'istanza non richiede compromessi tra fattibilità e obiettivo commerciale; in casi più grandi, invece, potrebbero emergere trade-off tra audience, equità e fatica.

5.3.4 Analisi dello schedule

Tabella 5.2: Schedule ottimo prodotto dal modello CP-SAT per l'istanza del 13–15 dicembre 2022.

Partita	Home	Away	Big match	Slot	Giorno	Finestra
G_22200427	LAC	PHX	No	S1	Tuesday	19:00 ESPN
G_22200426	UTA	NOP	No	S2	Tuesday	21:30 TNT
G_22200425	MEM	MIL	Sì	S3	Tuesday	22:00 ESPN
G_22200415	IND	GSW	No	S4	Wednesday	19:00 NBA TV
G_22200420	SAS	POR	No	S5	Wednesday	20:00 ESPN
G_22200424	HOU	MIA	Sì	S6	Wednesday	21:30 TNT
G_22200417	TOR	SAC	No	S7	Wednesday	22:00 ESPN
G_22200416	ORL	ATL	No	S8	Thursday	19:00 NBA TV
G_22200409	PHI	SAC	No	S9	Thursday	19:30 ESPN
G_22200410	MIL	GSW	Sì	S10	Thursday	21:00 TNT
G_22200413	LAL	BOS	Sì	S11	Thursday	21:30 ESPN
G_22200411	HOU	PHX	No	S12	Thursday	22:00 ABC
G_22200412	UTA	NOP	No	S13	Friday	19:00 ESPN

La Tabella 5.2 mostra la soluzione ottima in forma leggibile. Le quattro partite marcate come big match sono tutte assegnate a slot prime time: MEM–MIL, HOU–MIA, MIL–GSW e LAL–BOS. La partita LAL–BOS, storicamente ad alta attrattività, viene collocata giovedì alle 21:30 su ESPN, una scelta coerente con l’obiettivo di esposizione. Lo schedule non lascia partite non assegnate e non viola la capacità degli slot.

Lo schedule ottenuto non deve essere letto come calendario NBA reale alternativo, ma come risultato di una KB vincolata su un sottoinsieme reale di partite. La sua qualità deriva dalla soddisfazione simultanea di tre livelli: completezza, fattibilità e valore commerciale. La completezza è garantita dal vincolo di assegnazione; la fattibilità dalla capacità degli slot e dall’assenza di conflitti squadra-giorno; il valore commerciale dall’obiettivo sui big match.

La soluzione evidenzia anche un aspetto importante: gli slot prime time non sono saturati esclusivamente da big match. Alcune partite non marcate come big match entrano comunque in finestre pregiate perché l’inventario disponibile e i vincoli di giorno/canale devono essere rispettati. Questo comportamento è realistico: una rete televisiva non trasmette solo partite di massimo richiamo, ma deve distribuire contenuti su più canali e fasce.

Dal punto di vista della spiegabilità, ogni scelta può essere ricondotta a una regola. Se MEM–MIL è in prime time, la ragione è la sua appartenenza a M^B e il contributo positivo all’obiettivo. Se due partite non occupano lo stesso slot, la ragione è il vincolo di capacità. Se una squadra non viene programmata due volte nello stesso giorno, la ragione è il vincolo team-giorno. Questa tracciabilità è un vantaggio rispetto a euristiche opache.

5.4 CP-SAT e ragionamento

Google OR-Tools CP-SAT combina programmazione a vincoli, tecniche SAT e ottimizzazione intera. La propagazione dei vincoli riduce i domini: se una partita viene assegnata a uno slot, tutte le altre variabili della stessa partita e dello stesso slot vengono forzate a zero. Il branch-and-bound mantiene il miglior valore obiettivo noto e scarta rami che non possono migliorarlo. Le tecniche CDCL apprendono clausole dai conflitti, evitando di ripetere scelte già dimostrate impossibili.

5.4.1 Complessità del problema

Senza vincoli, ogni partita potrebbe essere assegnata a uno qualunque dei 14 slot, producendo 14^{13} assegnazioni possibili. La formulazione binaria introduce 182 variabili, quindi lo spazio booleano grezzo sarebbe 2^{182} configurazioni. I vincoli riducono lo spazio utile, ma il problema resta combinatorio.

La famiglia di vincoli di assegnazione e capacità crea una struttura simile a un matching bipartito, mentre i vincoli team-giorno e business introducono dipendenze aggiuntive. Il problema non è solo scegliere il miglior slot per ogni partita in modo indipendente: ogni scelta consuma risorse e modifica le scelte ammissibili per le altre partite.

In forma generale, problemi di scheduling con risorse limitate, vincoli temporali e funzione obiettivo appartengono alla famiglia dei problemi combinatori difficili e sono riconducibili a varianti NP-hard [12]. CP-SAT è scelto perché combina propagazione dei vincoli, ricerca branch-and-bound e tecniche SAT moderne.

5.4.2 Estensione a vincoli soft

Nel modello attuale i vincoli business sono trattati come hard constraint. Questa scelta semplifica l'interpretazione, ma in un contesto industriale può essere troppo rigida. Una rete televisiva potrebbe accettare una lieve sovrapposizione tra big match se ciò consente di evitare un conflitto logistico più grave. La versione successiva del modello può introdurre variabili di penalità:

$$violation_k \geq 0$$

e una funzione obiettivo multi-termine:

$$\max \text{prime_time_exposure} - \lambda_1 \text{overlap_penalty} - \lambda_2 \text{fatigue_penalty}$$

Questa formulazione trasformerebbe il modello in un Constraint Optimization Problem pienamente flessibile, capace di esprimere compromessi invece di risposte binarie fattibile/non fattibile.

5.5 Valutazione e discussione critica

Il risultato ottimo conferma che il modello è corretto sull'istanza scelta. Il modello assegna tutte le 13 partite, rispetta la capacità degli slot e colloca tutti i 4 big match disponibili in prime time, ottenendo valore obiettivo 4.

La validazione verifica tre proprietà: coerenza, completezza e ottimalità. La coerenza è data dall'assenza di slot con più di una partita e dall'assenza di conflitti squadra-giorno; la completezza è evidente dalla tabella dello schedule, in cui tutti i game id compaiono una volta; l'ottimalità è certificata dallo stato `OPTIMAL` e dal valore obiettivo pari al numero totale di big match disponibili.

La scelta di implementare le regole business come vincoli hard è conservativa. In uno scenario più flessibile, alcune regole commerciali potrebbero diventare soft constraint: violarle non renderebbe impossibile il calendario, ma introdurrebbe una penalità. Questa trasformazione convertirebbe il CSP in un COP capace di gestire casi in cui massimizzare audience ed equità sportiva non è simultaneamente possibile.

5.5.1 Limiti della modellazione CSP

Come ampliamento futuro, il modello potrebbe includere disponibilità reali delle arene, contratti televisivi effettivi, preferenze geografiche, vincoli di viaggio e finestre di riposo consecutive. Anche la definizione di big match potrebbe essere estesa oltre la win rate rolling, includendo rivalità storiche, presenza di superstar, mercato televisivo o implicazioni playoff.

Capitolo 6

Ricerca Informata e ottimizzazione logistica

6.1 Sommario

Il modulo di ricerca informata ottimizza un road trip NBA su 18 nodi, includendo il nodo Home e 17 destinazioni. Il problema è una variante del Traveling Salesman Problem: partire dalla base logistica, visitare ogni arena esattamente una volta e tornare alla base minimizzando un costo composto da distanza e costi fissi logistici. La ricerca informata è appropriata perché lo spazio degli stati è esponenziale, ma una buona euristica può guidare l'esplorazione verso soluzioni ottime senza enumerare tutte le permutazioni.

6.2 Fondamenti teorici della ricerca informata

La ricerca in spazi di stati formalizza un problema come insieme di stati, stato iniziale, azioni, funzione di transizione, test di goal e funzione di costo. Una soluzione è un cammino dallo stato iniziale a uno stato goal; una soluzione ottima minimizza il costo complessivo. Questa impostazione è tipica dell'Intelligenza Artificiale classica e permette di risolvere problemi decisionali anche quando non esiste una formula chiusa per la soluzione [13].

A* estende Uniform Cost Search introducendo una funzione euristica $h(n)$, cioè una stima del costo residuo fino al goal. La priorità diventa:

$$f(n) = g(n) + h(n)$$

Se $h(n)$ è ammissibile, cioè non sovrastima mai il costo reale residuo, A* è ottimale. Se $h(n)$ è anche consistente, cioè soddisfa:

$$h(n) \leq c(n, n') + h(n')$$

per ogni transizione da n a n' , allora i costi lungo i cammini sono monotoni e non è necessario riaprire stati già espansi [14]. La qualità di A* dipende quindi dalla qualità dell'euristica: una euristica nulla riduce A* a Uniform Cost Search; una euristica informativa riduce drasticamente l'esplorazione.

Nel problema del road trip NBA, la ricerca informata è adatta perché ogni stato parziale contiene decisioni già prese e città ancora da visitare. Una ricerca non informata

esplorerebbe molte sequenze equivalenti o chiaramente peggiori. L'euristica MST introduce conoscenza geometrica: per completare il tour bisogna comunque connettere le città rimanenti e tornare alla base.

6.3 Modellazione dello spazio degli stati

Uno stato è definito come:

$$n = (c, R)$$

dove c è la città corrente e R è l'insieme ordinato/hashabile delle città non ancora visitate. Lo stato iniziale è $(Home, C)$, con tutte le destinazioni ancora da visitare; uno stato goal ha $R = \emptyset$ e richiede il ritorno a Home. Il costo di transizione combina distanza Haversine e costo fisso della destinazione:

$$cost(i, j) = 1.5 \cdot dist_{hav}(i, j) + fixed(j)$$

Questa formulazione è più realistica di una distanza pura: spostarsi verso una città comporta anche costi logistici di arrivo, preparazione, gestione locale e permanenza.

6.4 Algoritmo A*

A* usa la funzione:

$$f(n) = g(n) + h(n)$$

dove $g(n)$ è il costo accumulato e $h(n)$ è una stima inferiore del costo residuo. La frontiera è ordinata per $f(n)$; a parità di costo, il sistema usa criteri deterministici per rendere riproducibile l'esplorazione. Il dizionario dei migliori $g(n)$ per stato evita di espandere percorsi dominati: se lo stesso stato è già stato raggiunto con costo minore, il nuovo percorso non può condurre a una soluzione migliore.

6.5 Euristica MST

Per un insieme residuo R , l'euristica è:

$$h(n) = MST(R) + \min_{r \in R} d(c, r) + \min_{r \in R} d(Home, r) + \sum_{r \in R} fixed(r)$$

Il termine $MST(R)$ è un lower bound sul costo necessario a connettere tutte le città rimanenti. Il collegamento minimo dalla città corrente rappresenta il costo minimo inevitabile per entrare nel sottoinsieme non visitato; il collegamento minimo verso Home rappresenta il costo minimo inevitabile per chiudere il tour. I costi fissi sono obbligatori una volta per destinazione e quindi possono essere sommati senza dipendere dall'ordine.

L'euristica è ammissibile perché ogni tour completo deve connettere i nodi rimanenti, deve raggiungerli dalla posizione corrente e deve tornare a Home. Ciascun termine è una stima per difetto di una parte necessaria del completamento. È anche consistente: visitando una città, il relativo costo fisso viene trasferito da h a g , mentre il lower bound sui nodi rimanenti non aumenta in modo da violare la disuguaglianza triangolare. Questo consente ad A* di non riaprire stati già chiusi.

6.5.1 Analisi dell'euristica

L'ammissibilità dell'euristica MST deriva dal fatto che ogni tour Hamiltoniano che visita tutte le città rimanenti contiene implicitamente una struttura connessa su tali città. Rimuovendo archi da un tour si ottiene un albero o una struttura connessa; il Minimum Spanning Tree è per definizione la struttura connessa di costo minimo. Quindi il costo dell'MST non può superare il costo di qualunque completamento valido.

I due termini di connessione $\min d(c, r)$ e $\min d(Home, r)$ sono anch'essi lower bound: il percorso deve entrare nell'insieme residuo partendo dalla città corrente e deve uscire o chiudersi verso Home. Usare il collegamento minimo produce una stima ottimistica, quindi sicura. I costi fissi sono obbligatori una volta per destinazione, dunque possono essere sommati esattamente.

Rispetto a euristiche alternative, la MST è più informativa della sola distanza al vicino più prossimo, perché considera la struttura globale dei nodi rimanenti. È anche meno costosa di un rilassamento esatto tipo Held-Karp su ogni stato. Una euristica basata solo su nearest neighbor sarebbe veloce ma più debole; una euristica basata su soluzione TSP rilassata sarebbe più forte ma computazionalmente troppo onerosa. La MST rappresenta quindi un compromesso efficace tra qualità del lower bound e costo di calcolo.

6.6 Ottimizzazioni

Il modulo usa tre ottimizzazioni fondamentali. La prima è la memoizzazione con `lru_cache` per MST ed euristica: molti stati condividono lo stesso insieme di città residue e sarebbe inefficiente ricalcolare ogni volta l'albero ricoprente minimo. La seconda è l'upper bound iniziale ottenuto con nearest neighbor e migliorato con 2-opt: anche se non garantisce ottimalità, fornisce subito una soluzione ammissibile contro cui potare rami peggiori. La terza è il branch-and-bound: ogni stato con $f(n)$ non inferiore all'incumbent viene scartato.

Tabella 6.1: Risultati quantitativi dell'ottimizzazione A* per il road trip NBA su 18 nodi.

Metrica	Valore
Nodi totali inclusa Home	18
Destinazioni da visitare	17
Spazio teorico degli stati	2 359 296
Costo ottimo	\$684 107.41
Tempo di esecuzione	336.20 ms
Stati espansi	528
Stati inseriti in frontiera	557
Transizioni/stati potati	5 119
Stati obsoleti scartati	15
Upper bound iniziale NN + 2-opt	\$706 474.31
Gap upper bound iniziale	3.27%

I risultati mostrano la forza della ricerca informata. Lo spazio teorico contiene 2 359 296 stati, ma A* ne espande solo 528. Il costo ottimo è \$684 107.41, mentre la soluzione iniziale nearest neighbor + 2-opt vale \$706 474.31: il gap iniziale è 3.27%. Questo significa che il greedy migliorato produce già una buona soluzione, ma A* è necessario per certificare l'ottimalità e recuperare un risparmio non trascurabile.

Tabella 6.2: Sequenza della rotta ottima trovata da A* per il road trip NBA.

Ordine	Città visitata
0	Home (Chase Center, San Francisco)
1	Denver
2	Memphis
3	Chicago
4	Milwaukee
5	Toronto
6	Boston
7	New York
8	Philadelphia
9	Washington DC
10	Charlotte
11	Atlanta
12	Miami
13	New Orleans
14	Houston
15	Dallas
16	Phoenix
17	Los Angeles
18	Home

La Tabella 6.2 mostra una rotta geograficamente interpretabile: dalla base di San Francisco il percorso procede verso l'interno, attraversa il Midwest, raggiunge la costa Est, scende verso il Sud-Est, rientra in Texas e chiude con Phoenix e Los Angeles. La sequenza evita salti ripetuti tra coste opposte e costruisce un tour coerente con l'obiettivo di ridurre distanza e costi fissi.

6.6.1 Confronto con strategie non informate

Una ricerca in ampiezza non è adatta perché considera tutti i percorsi parziali allo stesso livello senza distinguere quelli promettenti. Uniform Cost Search garantirebbe ottimalità con costi positivi, ma senza euristica esplorerebbe molti più stati. Nearest Neighbor è veloce, ma non garantisce ottimalità: sceglie sempre la prossima tappa meno costosa e può lasciare alla fine collegamenti molto costosi. La variante 2-opt migliora localmente la rotta, ma resta intrappolabile in minimi locali.

A* occupa una posizione intermedia: mantiene la garanzia di ottimalità ma usa conoscenza euristica per ordinare la frontiera.

Tabella 6.3: Effetto della memoizzazione sulle funzioni MST ed euristica nel modulo A*.

Funzione memoizzata	Hit	Miss	Hit rate
MST sui nodi rimanenti	1 148	3 516	24.61%
Euristica A* completa	1 011	4 665	17.81%

Le cache non sono un dettaglio implementativo secondario: rendono operativo il calcolo ripetuto di lower bound su sottoinsiemi. L'hit rate dell'MST (24.61%) e dell'euristica

(17.81%) indica che una parte significativa della ricerca riusa conoscenza già calcolata. In termini KBS, la cache agisce come memoria di inferenze parziali: stati diversi possono condividere la stessa sottostruttura residua.

6.6.2 Analisi del pruning

Il pruning è il meccanismo che trasforma una formulazione esponenziale in una procedura praticabile sull'istanza considerata. Il solver scarta 5119 transizioni o stati perché il loro lower bound non può migliorare la soluzione incumbent. In pratica, A^* non deve dimostrare esplicitamente che ogni permutazione è peggiore: è sufficiente dimostrare che intere famiglie di completamenti hanno un costo minimo teorico già superiore al miglior tour noto.

Il rapporto tra stati teorici ed espansi è particolarmente significativo. Espandere 528 stati su 2359296 equivale a visitare una frazione minima dello spazio. Questo dato non elimina la complessità esponenziale nel caso pessimo, ma dimostra che la conoscenza euristica e l'upper bound iniziale sono efficaci sulla struttura geografica scelta.

6.6.3 Interpretazione logistica

Il costo ottimo non rappresenta solo chilometri. La funzione usata somma una componente proporzionale alla distanza e una componente fissa per destinazione. Questa scelta riflette un principio logistico realistico: ogni trasferta ha costi legati al viaggio e costi legati alla gestione locale. Due rotte con distanza simile possono quindi avere costi diversi se visitano città con costi fissi differenti in posizioni diverse del tour.

Nel dominio NBA, tale modello può supportare decisioni come raggruppare partite geograficamente vicine, evitare rientri inutili e stimare la difficoltà di un road trip. L'uso diretto di questi costi dentro il CSP non è implementato nel progetto attuale e viene quindi trattato come sviluppo futuro, non come componente già realizzata.

6.7 Valutazione e discussione critica

Il risultato principale del modulo A^* è la certificazione di una rotta ottima sull'istanza considerata. A fronte di 2359296 stati teorici, l'algoritmo espande 528 stati e trova un costo ottimo pari a \$684107.41. Il confronto con la soluzione iniziale nearest neighbor + 2-opt, pari a \$706474.31, mostra che l'euristica iniziale fornisce già una rotta valida, ma non ottima.

La differenza tra spazio teorico e stati effettivamente espansi indica che l'euristica MST e il branch-and-bound riducono in modo sostanziale la ricerca necessaria. Il modulo non si limita quindi a produrre una rotta plausibile: restituisce una soluzione ottima rispetto alla funzione di costo implementata e documenta il risparmio rispetto alla soluzione euristica iniziale.

6.7.1 Limiti metodologici e sviluppi

Come ampliamento futuro, il modello potrebbe includere disponibilità delle arene, giorni di riposo, fusi orari, voli charter, sequenze di back-to-back e possibilità di rientro intermedio. Anche l'euristica MST potrebbe essere integrata con vincoli temporali o preferenze di

calendario, così da rendere la stima del costo residuo più vicina alle condizioni operative reali.

Un'estensione naturale consiste nel calcolare road trip per singola squadra e trasformare il costo ottenuto in una feature di fatigue o in una penalità dello scheduling. Questo collegamento tra ML, CSP e A^* è però da considerare sviluppo futuro: nella versione attuale i moduli condividono l'impostazione progettuale, ma non esiste ancora un ciclo end-to-end implementato.

Capitolo 7

Conclusioni

Il progetto ha costruito una pipeline modulare nel dominio NBA, partendo dalla raccolta e pulizia dei dati fino alla modellazione predittiva, alla pianificazione con vincoli e alla ricerca informata. La fase di assessment e cleaning ha prodotto una base dati coerente, successivamente analizzata tramite EDA. I controlli sui grafici hanno verificato il bilanciamento tra W e L, tra osservazioni Home e Away, la presenza di outlier prestazionali e correlazioni rilevanti tra statistiche tecniche.

Il feature engineering ha trasformato record squadra-partita in esempi pre-partita, usando rolling windows, differenziali home-away, indicatori di riposo, back-to-back, streak e normalizzazioni stagionali. Questa rappresentazione ha permesso di affrontare due task supervisionati: classificazione di `home_win` e regressione di `point_differential`. Nei risultati finali, tutti i modelli superano la baseline maggioritaria; Gradient Boosting ottiene la migliore accuracy di classificazione sul test set, mentre XGBoost risulta il migliore nella regressione del differenziale punti.

Il modulo CSP ha formalizzato lo scheduling televisivo come problema di assegnazione con vincoli hard, vincoli business e funzione obiettivo. Sull'istanza considerata, composta da 13 partite, 14 slot e 4 big match, il solver restituisce una soluzione **OPTIMAL**: tutte le partite sono assegnate, non ci sono violazioni dei vincoli e tutti i big match vengono collocati in prime time, ottenendo valore obiettivo 4.

Il capitolo sulla ricerca informata ha mostrato l'ottimizzazione di un road trip NBA tramite A* con euristica MST. Lo spazio teorico contiene 2 359 296 stati, ma l'algoritmo espande 528 stati e certifica una rotta ottima con costo pari a \$684 107.41. Il confronto con nearest neighbor + 2-opt evidenzia che la soluzione euristica iniziale è valida ma non ottima, mentre A* fornisce certificazione formale rispetto alla funzione di costo implementata.

Nel complesso, i risultati ottenuti mostrano un sistema coerente: i dati vengono puliti e trasformati in conoscenza utilizzabile, i modelli predittivi apprendono dai segnali storici, il CSP produce uno schedule fattibile e ottimo sull'istanza definita, e A* risolve un problema logistico separato con garanzia di ottimalità.

7.1 Sviluppi Futuri

ML e CSP condividono il dataset feature engineered, mentre A* fornisce una dimostrazione separata di ottimizzazione logistica. Negli schemi iniziali, le frecce tratteggiate indicano collegamenti progettuali non ancora automatizzati. Il passo successivo è chiudere il ci-

clo: usare predizioni e costi logistici per generare calendari, poi rivalutare l'impatto del calendario sulle predizioni.

Gli sviluppi naturali includono: integrazione CSP→ML per stimare l'impatto competitivo del calendario generato; modellazione esplicita della travel fatigue; vincoli realistici del calendario NBA; reti bayesiane per incertezza su infortuni, forma e audience; reasoning probabilistico sui big match; dashboard interattive per esplorare scenari di scheduling.

Bibliografia

- [1] Fayyad, Piatetsky-Shapiro e Smyth, From Data Mining to Knowledge Discovery in Databases, AI Magazine, 1996.
- [2] Tukey, Exploratory Data Analysis, Addison-Wesley, 1977.
- [3] Bishop, Pattern Recognition and Machine Learning, Springer, 2006.
- [4] Hastie, Tibshirani e Friedman, The Elements of Statistical Learning, Springer, 2009.
- [5] Breiman, Random Forests, Machine Learning, 2001.
- [6] Friedman, Greedy Function Approximation: A Gradient Boosting Machine, Annals of Statistics, 2001.
- [7] `nba_api`, documentazione e endpoint per dati NBA.
- [8] Pedregosa et al., Scikit-learn: Machine Learning in Python, JMLR, 2011.
- [9] Chen e Guestrin, XGBoost: A Scalable Tree Boosting System, KDD, 2016.
- [10] Google OR-Tools, CP-SAT solver documentation.
- [11] Hagberg, Schult, Swart, NetworkX, SciPy, 2008.
- [12] Dechter, Constraint Processing, Morgan Kaufmann, 2003.
- [13] Russell e Norvig, Artificial Intelligence: A Modern Approach, Pearson.
- [14] Hart, Nilsson e Raphael, A Formal Basis for the Heuristic Determination of Minimum Cost Paths, IEEE Transactions on Systems Science and Cybernetics, 1968.